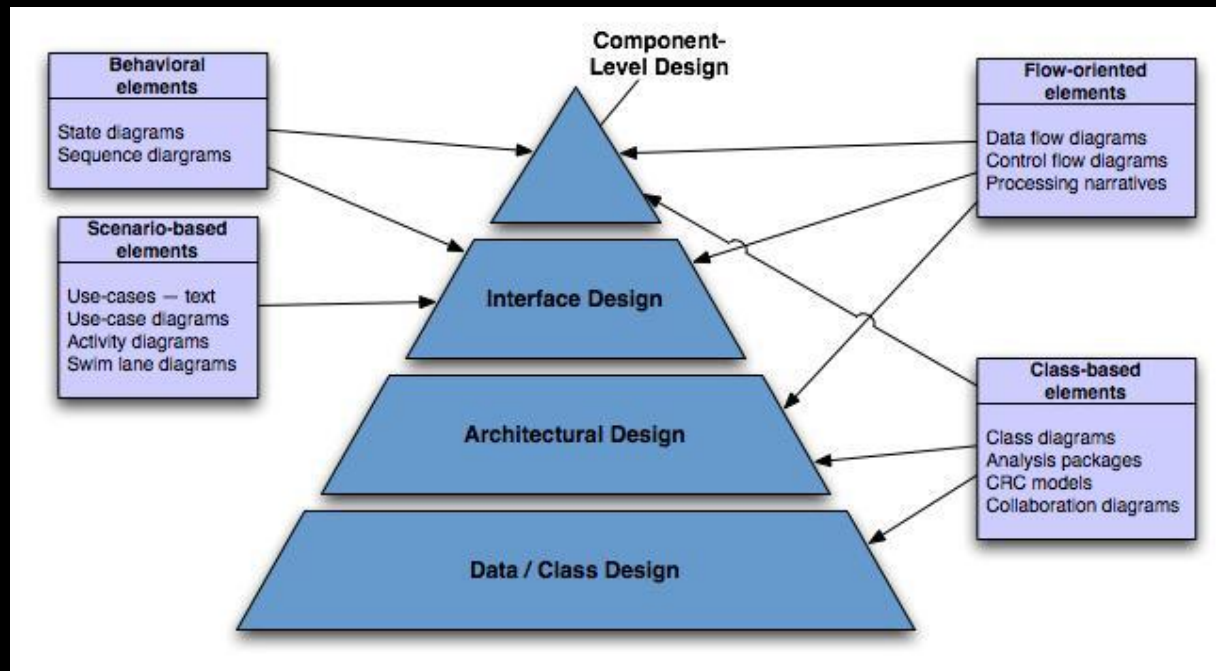


Chapter 3

Design Engineering

Analysis → Design



Design Concepts

1. Abstraction — data, procedure, control.
2. Architecture — the overall structure of the software.
3. Patterns — “conveys the essence” of a proven design solution.
4. Modularity — compartmentalization of data and function.
5. Information hiding — controlled interfaces.
6. Functional independence — high cohesion and low coupling.
7. Refinement — elaboration of detail for all abstractions.
8. Refactoring — improve design without effecting behavior.

Abstraction et al.

Abstraction

process – extracting essential details.

entity – a model or focused representation.

Information hiding

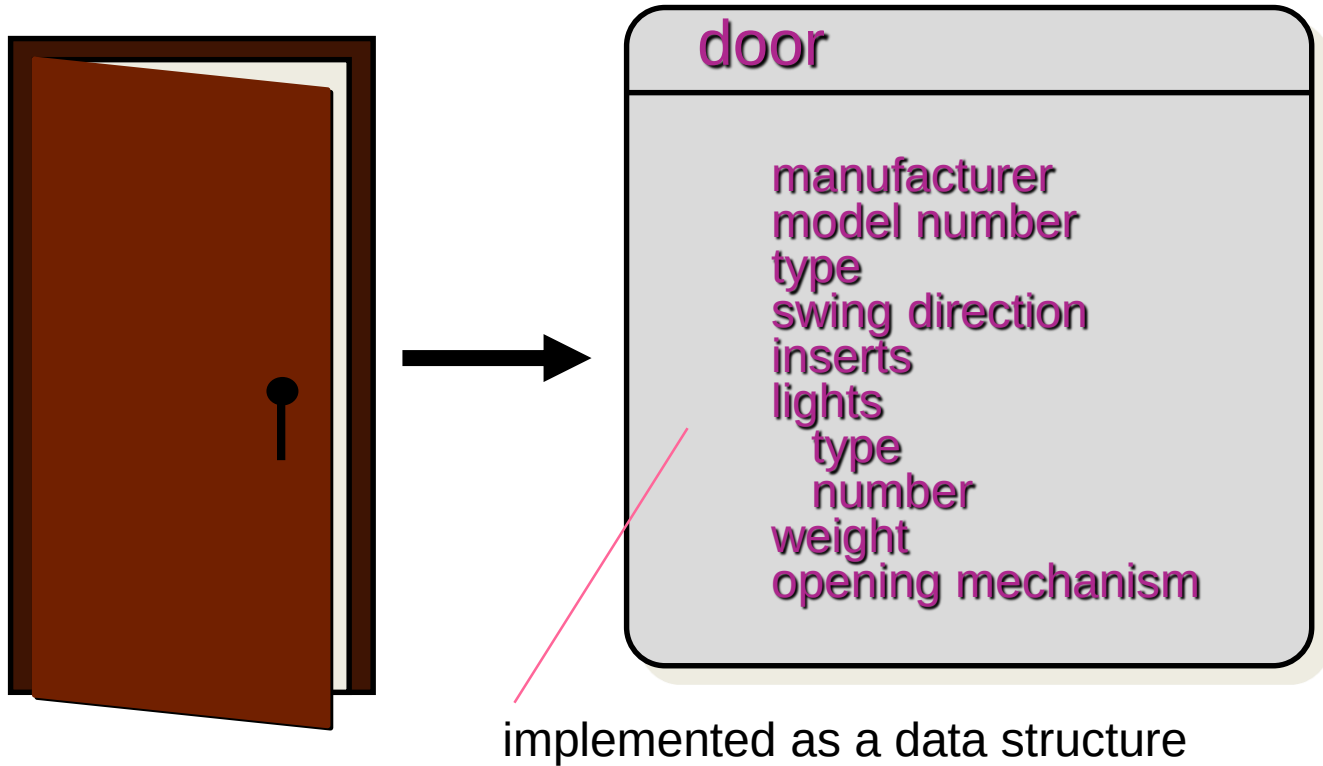
the suppression of inessential information.

Encapsulation

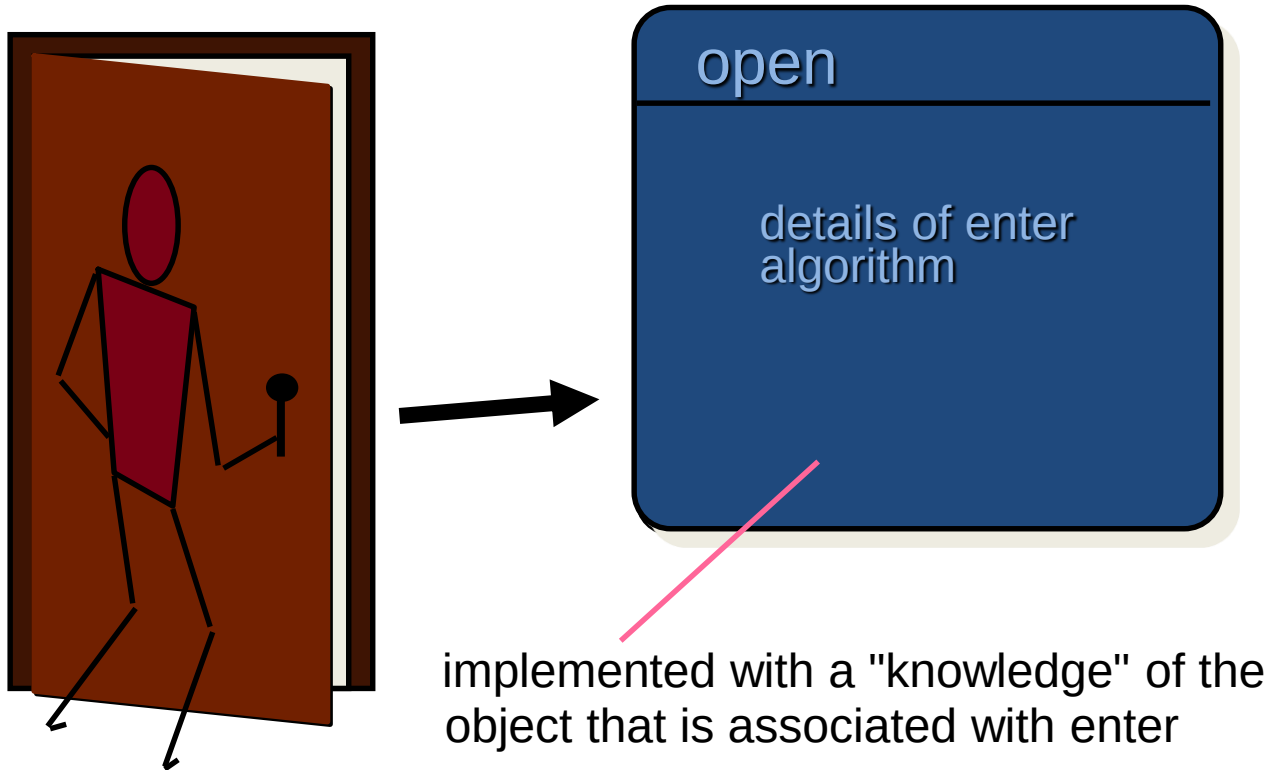
process – enclosing items in a container.

entity – enclosure that holds the items.

Data Abstraction



Procedural Abstraction



Architecture

“The overall structure of the software and the ways in which that structure provides conceptual integrity for a system.”

Structural properties: This aspect of the architectural design representation defines the components of a system (e.g., modules, objects, filters) and the manner in which those components are packaged and interact with one another.

Ex: objects are packaged to encapsulate both data and the processing that manipulates the data and interact via the invocation of methods. (Contd...)

Architecture(contd...)

Extra-functional properties: The architectural design description should address

- how the design architecture achieves requirements for performance,
- capacity,
- reliability,
- security,
- adaptability, and
- other system characteristics.

(Contd...)

Architecture(Contd...)

Families of related systems: The architectural design should draw upon **repeatable patterns** that are commonly encountered in the design of families of similar systems.

- In essence, the **design should have the ability to reuse architectural building blocks.**

Patterns

Pattern: Describes a particular recurring design problem that arises in specific design contexts, and presents a well-proven generic scheme for its solution.

Solution Scheme:

Describes its constituent components, their responsibilities & relationships, and the ways in which they collaborate.

Patterns

Design Pattern Template:

1. Pattern name—describes the essence of the pattern in a short but expressive name.

2. Intent—describes the pattern and what it does.

3. Also-known-as—lists any synonyms for the pattern.

4. Motivation—provides an example of the problem.

5. Applicability—notes specific design situations in which the pattern is applicable.

6.Structure—describes the classes that are required to implement the pattern.

7.Participants—describes the responsibilities of the classes that are required to implement the pattern.

8.Collaborations—describes how the participants collaborate to carry out their responsibilities.

9.Consequences—describes the “design forces” that affect the pattern and the potential trade-offs that must be considered when the pattern is implemented.

10.Related patterns—cross-references related design patterns.

Design Patterns

The best designers in any field have an uncanny(strangely familiar) ability to see patterns that characterize a problem and corresponding patterns that can be combined to create a solution.

- A description of a **design pattern** may also consider a **set of design forces**.

Design forces: describe **non-functional requirements** (e.g., ease of maintainability, portability) associated with the software for which the pattern is to be applied.

Pattern characteristics (classes, responsibilities, and collaborations): indicate the **attributes of the design** that may be adjusted to enable the pattern to accommodate a variety of problems.

Patterns

1) Architectural pattern: expresses a fundamental structural organization schema for software systems.

- **It provides:**
 - set of predefined subsystems,
 - their responsibilities, and
 - includes rules & guidelines for organizing the relationships between them.

2) Design pattern: Provides a scheme for refining the subsystems or components of a software system, or the relationships between them.

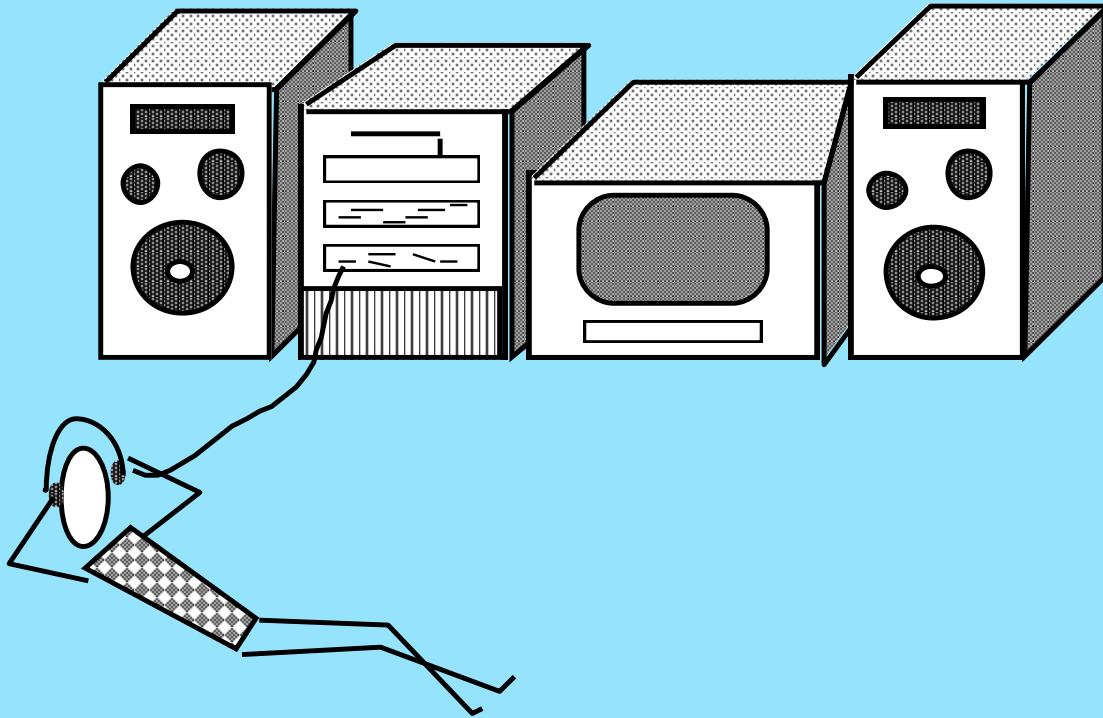
- It describes a commonly-recurring structure of communicating components that solves a general design problem within a particular context.

3) Idiom: is a low-level pattern specific to a programming language.

- An idiom describes how to implement particular aspects(specific feature) of components or the relationships between them using the features of the given language.

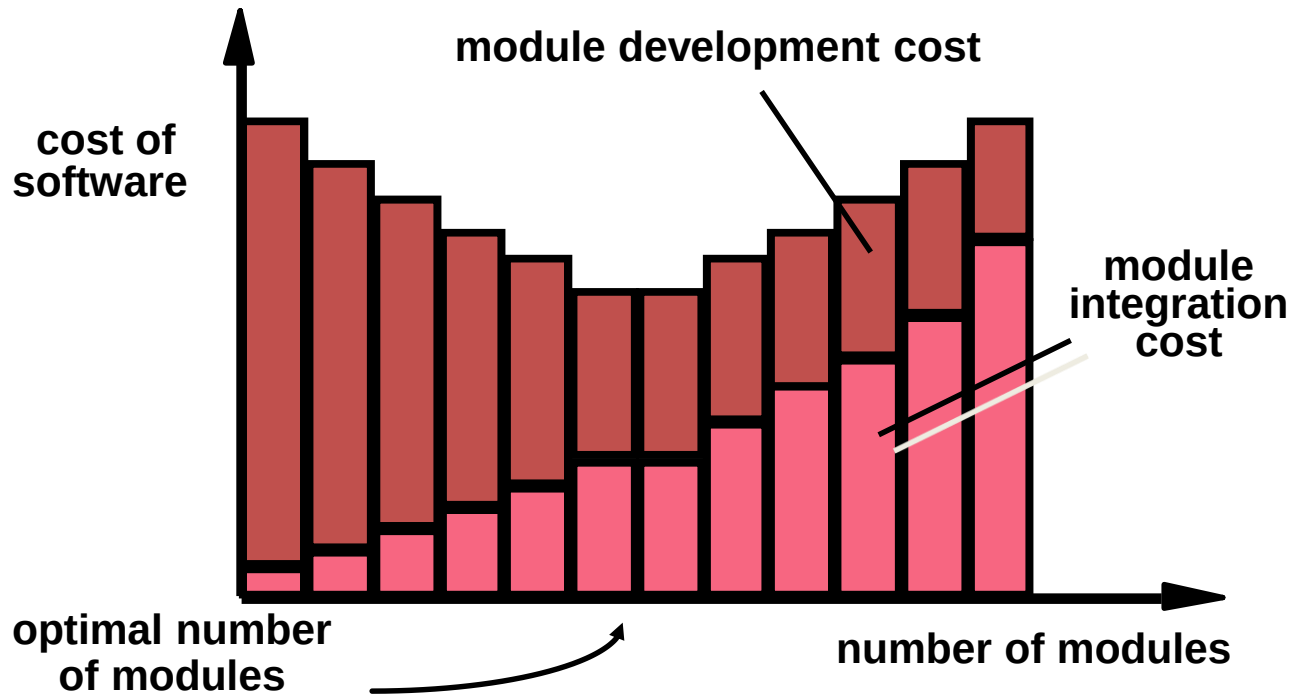
Modular Design

easier to build, easier to change, easier to fix ...

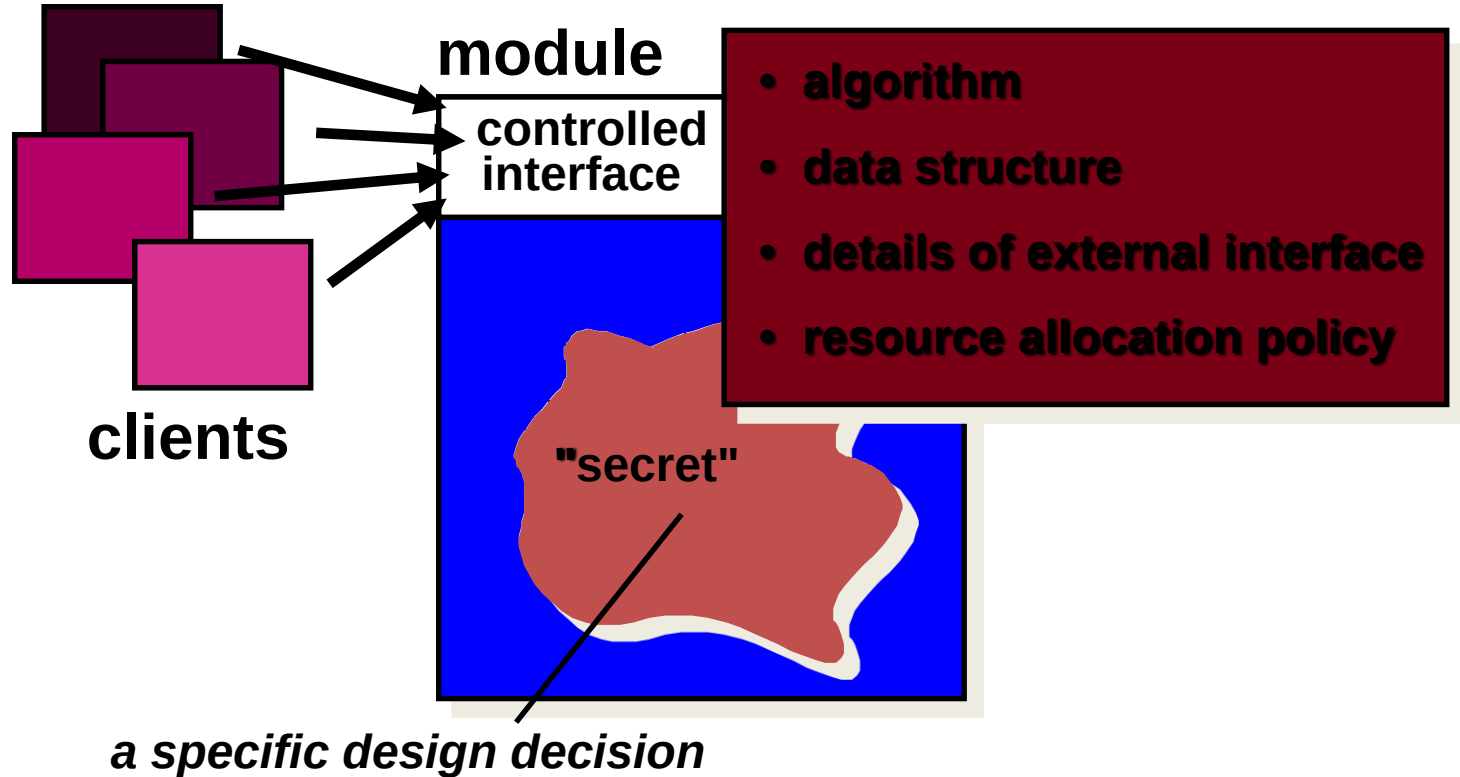


Modularity: Trade-offs

What is the "right" number of modules for a specific software design?



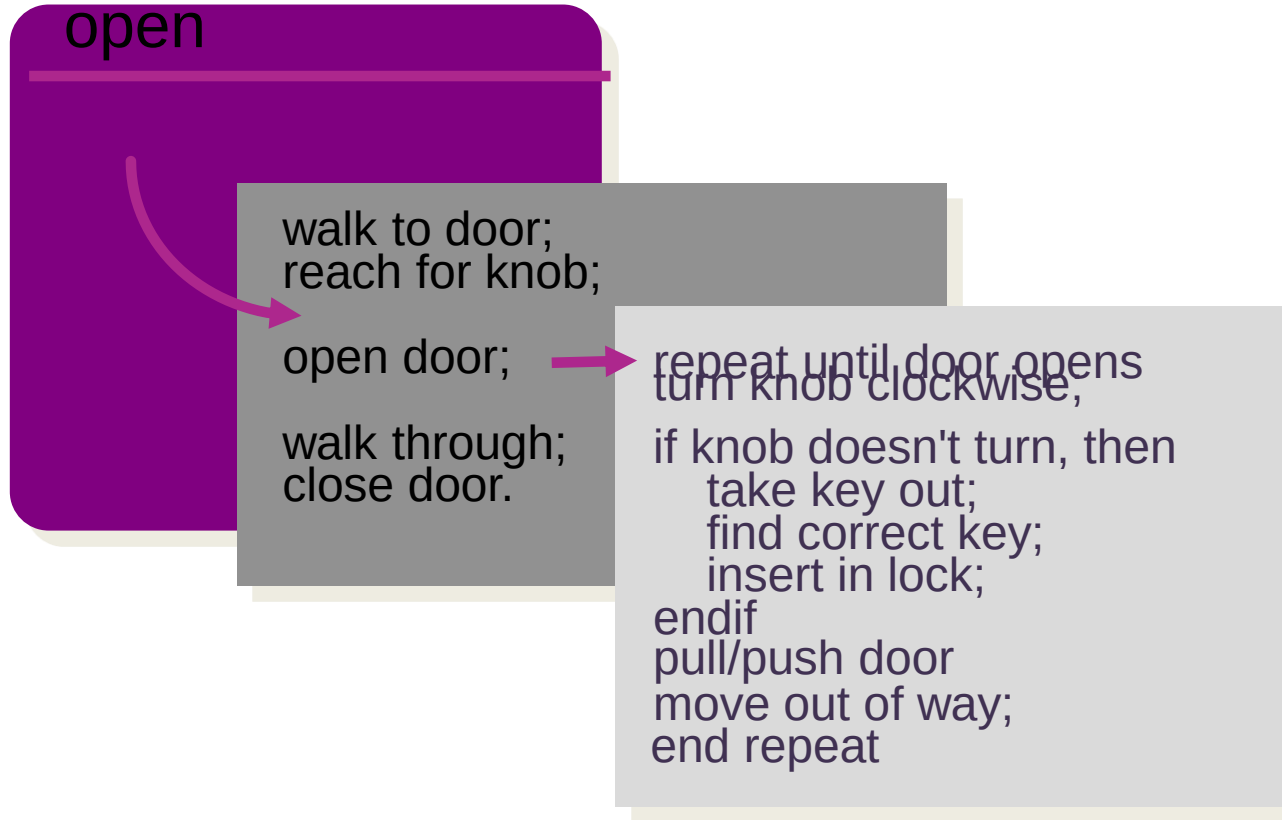
Information Hiding



Why Information Hiding?

- Reduces the likelihood of “**side effects**”.
- Limits the **global impact** of **local design decisions**.
- Emphasizes communication through **controlled interfaces**.
- **Discourages** the use of **global data**.
- Leads to **encapsulation**—an attribute of high quality design.
- Results in **higher quality software**.

Stepwise Refinement



Functional Independence

COHESION - the degree to which a module performs one and only one function.

COUPLING - the degree to which a module is "connected" to other modules in the system.

Refactoring

Fowler [FOW99] defines refactoring in the following manner:

"Refactoring is the process of changing a software system in such a way that it does not alter the external behavior of the code [design] yet improves its internal structure."

When software is refactored, the existing design is examined for:

- 1) redundancy**
- 2) unused design elements**
- 3) inefficient or unnecessary algorithms**
- 4) poorly constructed or inappropriate data structures**
- 5) or any other design failure that can be corrected to yield a better design.**

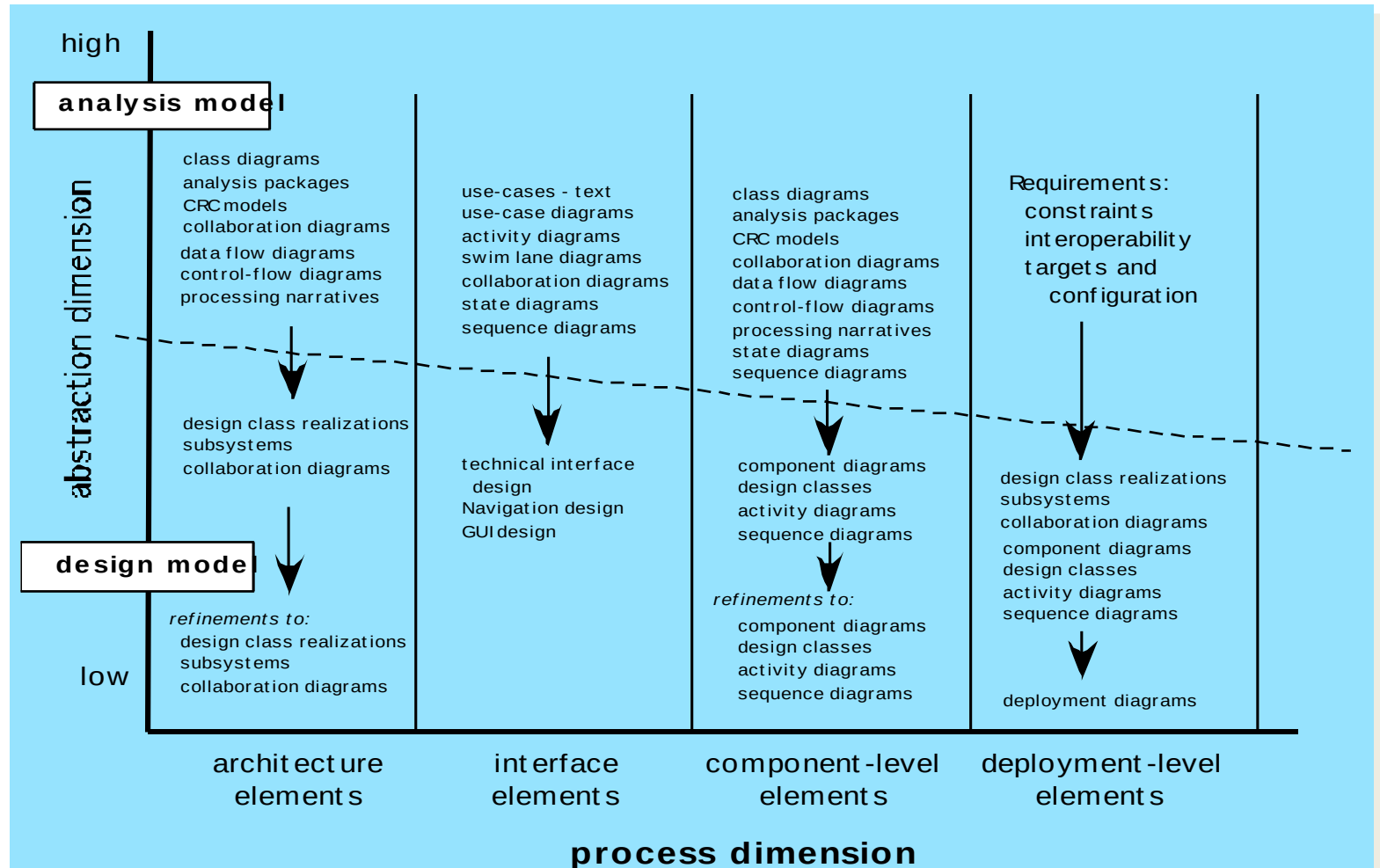
Design Classes(5)

- 1) **User interface classes** – define abstractions necessary for HCI.
- 2) **Business domain classes** – refinements of analysis classes.
- 3) **Process classes** – lower-level business abstractions that manage business domain classes.
- 4) **Persistent classes** – data stores (databases) that persist beyond execution of the software.
- 5) **System classes** – management and control functions that enable the system to operate and communicate within its computing environment and with the outside world.

Well-formed Design Class

- Complete and sufficient – class should be a complete and sufficient **encapsulation of reasonable attributes and methods**.
- Primitiveness – each method should be **focused on one thing**.
- High cohesion – class should be **focused on one kind of thing**.
- Low coupling – collaboration should be kept to an acceptable minimum.

The Design Model



Design Model Elements(5)

1) Data elements

Architectural level → databases and files

Component level → data structures

2) Architectural elements

An architectural model is derived from:

- i) Application domain
- ii) Analysis model
- iii) Available styles and patterns

3) Interface elements

There are three parts to the interface design element:

- i) The user interface (UI)
- ii) Interfaces to external systems
- iii) Interfaces to components within the application

4) Component elements

5) Deployment elements

Interface Elements

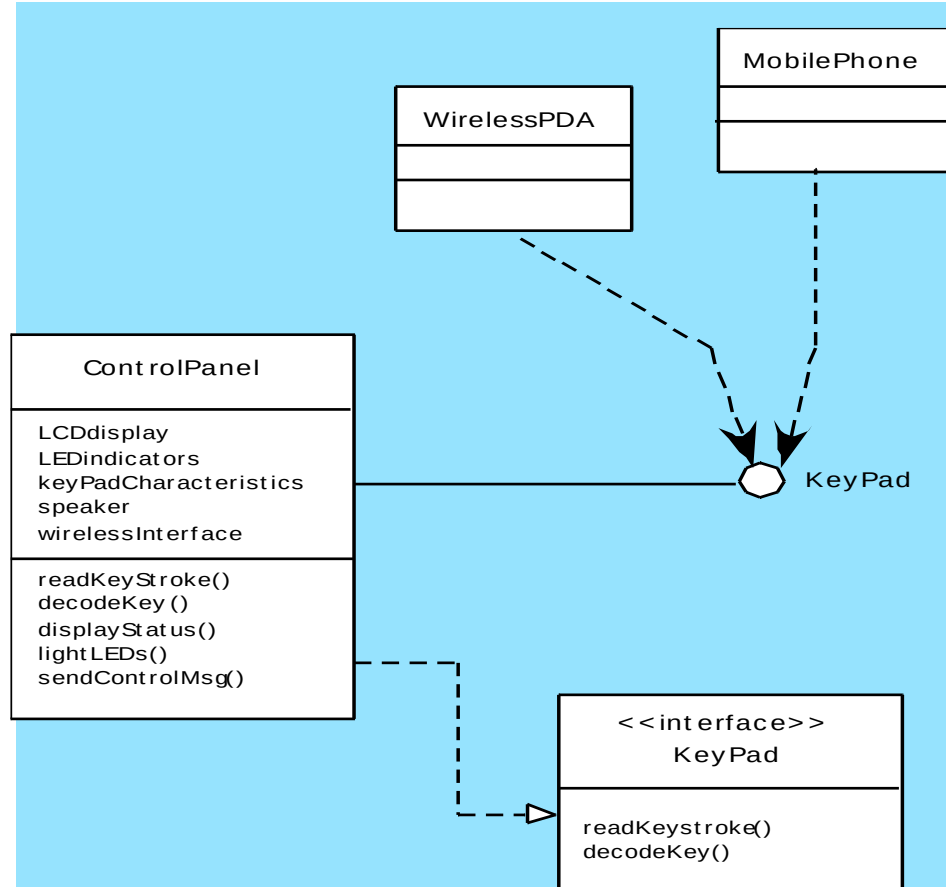
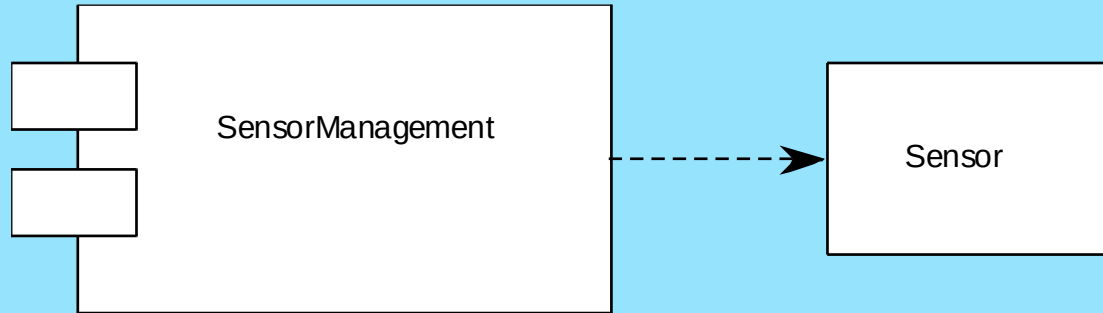
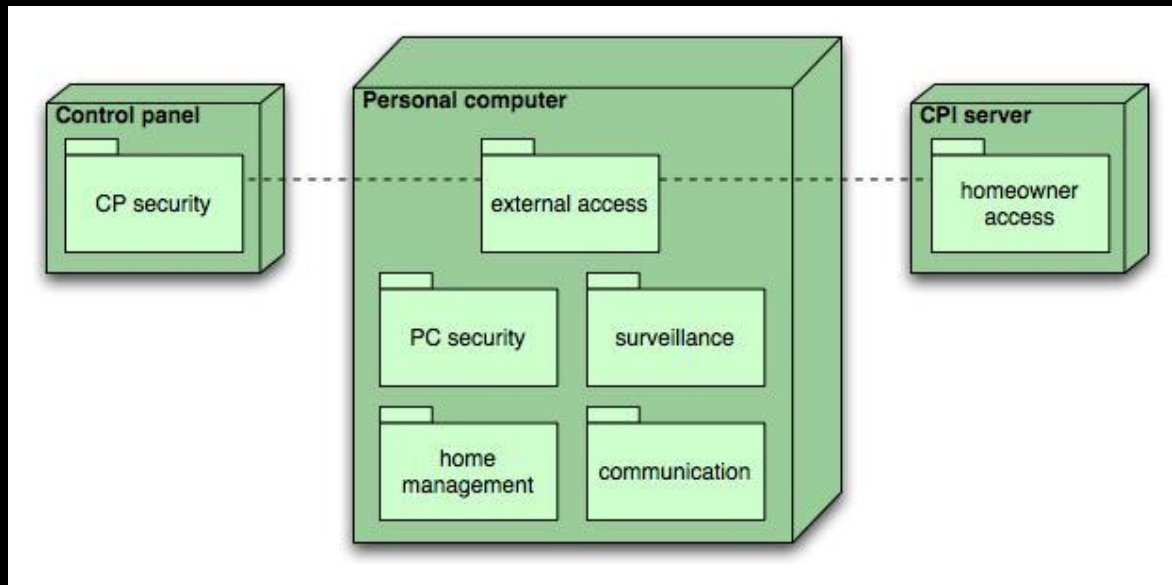


Figure 9.6 UML interface representation for **ControlPanel**

Component Elements



Deployment Diagram



Design Principles(10)

1)The design process should not suffer from ‘tunnel vision.’

- A good designer should consider alternative approaches, judging each based on the requirements of the problem,
- the resources available to do the job.

2)The design should be traceable to the analysis model.

- Because a single element of the design model can often be traced back to multiple requirements, it is necessary to have a means for tracking how requirements have been satisfied by the design model.

3)The design should not reinvent the wheel

- Systems are constructed using a set of design patterns, many of which have likely been encountered before.
- These patterns should always be chosen as an alternative to reinvention.
- Time is short and resources are limited; design time should be invested in representing truly new ideas and integrating patterns that already exist when applicable

4)The design should “minimize the intellectual distance” between the software and the problem as it exists in the real world.

- That is, the structure of the software design should, whenever possible, mimic the structure of the problem domain

5)The design should exhibit uniformity and integration.

- A design is uniform if it appears fully coherent. In order to achieve this outcome, rules of style and format should be defined for a design team before design work begins.
- A design is integrated if care is taken in defining interfaces between design components

6)The design should be structured to accommodate change.

- The design concepts discussed in the next section [abstraction, refinement, modularity, ...] enable a design to achieve this principle.

.

7)The design should be structured to degrade gently, even when aberrant data, events, or operating conditions are encountered.

-Well- designed software should never “bomb”; it should be designed to accommodate unusual circumstances, and if it must terminate processing, it should do so in a graceful manner

8)Design is not coding, coding is not design

-Even when detailed procedural designs are created for program components, the level of abstraction of the design model is higher than the source code.

-The only design decisions made at the coding level should address the small implementation details that enable the procedural design to be coded

9)The design should be assessed for quality as it is being created, not after the fact

-A variety of design concepts and design measures are available to assist the designer in assessing quality throughout the development process

10)The design should be reviewed to minimize conceptual (semantic) errors

-There is sometimes a tendency to focus on minutiae when the design is reviewed, missing the forest for the trees. -A design team should ensure that major conceptual elements of the design (omissions, ambiguity,

Design and Quality

- The design must implement all of the explicit requirements contained in the analysis model, and it must accommodate all of the implicit requirements desired by the customer.
- The design must be a readable, understandable guide for those who generate code and for those who test and subsequently support the software.
- The design should provide a complete picture of the software, addressing the data, functional, and behavioral domains from an implementation perspective.

Quality Guidelines(8)

1) A design should exhibit an architecture that

- (1) has been created using recognizable architectural styles or patterns,
- (2) is composed of components that exhibit good design characteristics and
- (3) can be implemented in an evolutionary fashion

For smaller systems, design can sometimes be developed linearly.

2) A design should be modular; that is, the software should be logically partitioned into elements or subsystems.

3) A design should contain distinct representations of data, architecture, interfaces, and components.

4) A design should lead to data structures that are appropriate for the classes to be implemented and are drawn from recognizable data patterns.

5) **A design should lead to components** that exhibit independent functional characteristics.

6) **A design should lead to interfaces** that reduce the complexity of connections between components and with the external environment.

7) **A design should be derived** using a repeatable method that is driven by information obtained during software requirements analysis.

8) **A design should be represented** using a notation that effectively communicates its meaning.

Chapter 11

Component-Level Design

Software Engineering: A Practitioner's Approach, 6th edition
by Roger S. Pressman

What is Comp. Level Design?

- A complete set of software components is defined during architectural design.
- But the **internal data structures** and **processing details** of each component are not represented at a level of abstraction that is close to code.
- **Component-level design defines the data structures, algorithms, interface characteristics, and communication mechanisms allocated to each component.**

What is a component?

- “A modular, deployable, and replaceable part of a system that encapsulates implementation and exposes a set of interfaces.”

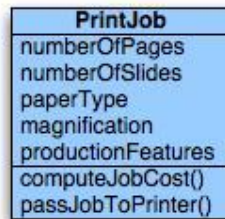
— *OMG UML Specification*

Component Views

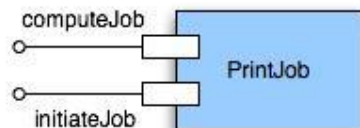
- OO View – A component is a **set of collaborating classes**.
- Conventional View – A component is a functional element of a program that incorporates **processing logic**, the **internal data structures** required to implement the **processing logic**, and an **interface** that enables the component to be invoked and data to be passed to it.

Class Elaboration

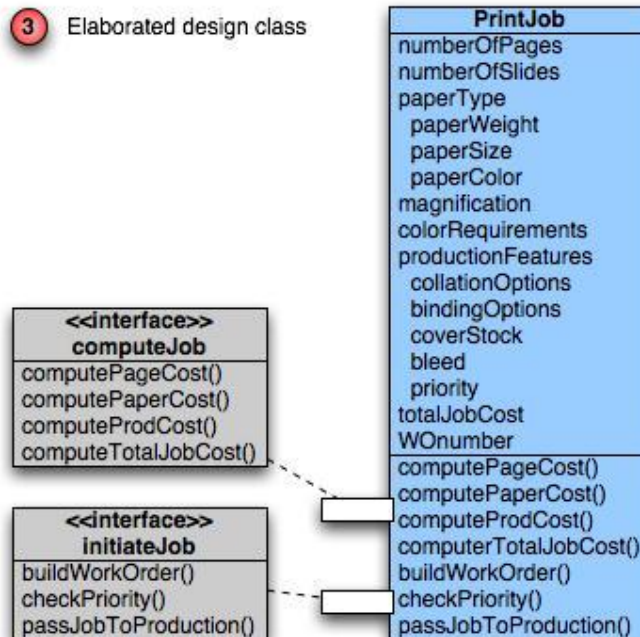
1 Analysis class



2 Design component



3 Elaborated design class



Design Principles(5)

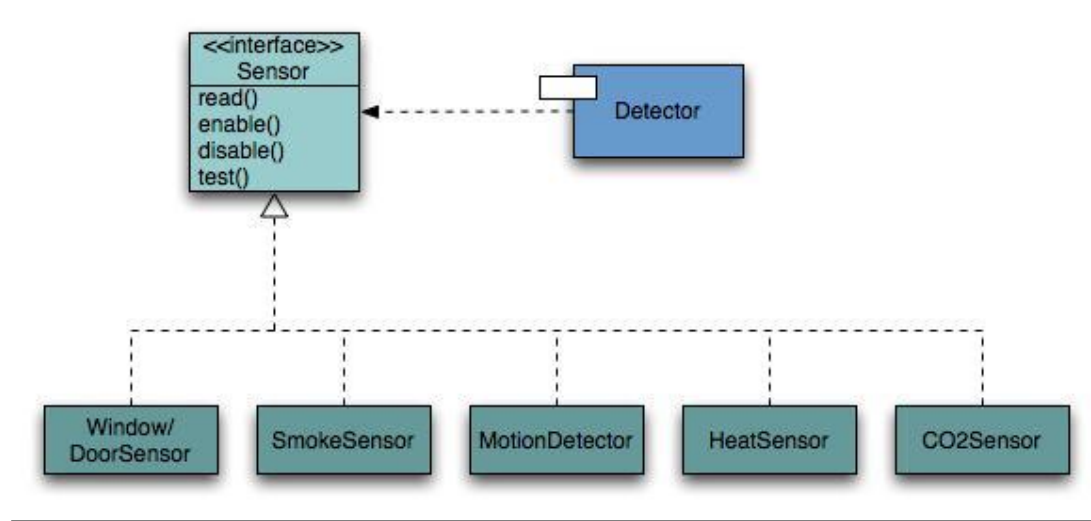
- 1) Design by Contract.**
- 2) Open-Closed Principle.**
- 3) Subtype Substitution.**
- 4) Depend on Abstractions.**
- 5) Interface Segregation.**

1.Design by Contract

- The relationship between a class and its clients can be viewed as a formal agreement, expressing each party's rights and obligations.
- Consider the following list operation:
- **public** Item remove(**int** index)
 - **requires** the specified index is in range ($0 \leq \text{index} < \text{size}()$).
 - **ensures** the element at the specified position in this list is removed, subsequent elements are shifted to the left (1 is subtracted from their indices), and the element that was removed is returned.

2.Open-Closed Principle

- A module should be open for extension but closed for modification.



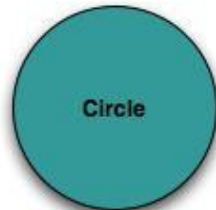
3.Substitutability

- Subclasses should be substitutable for base classes

Is a circle a kind of ellipse?



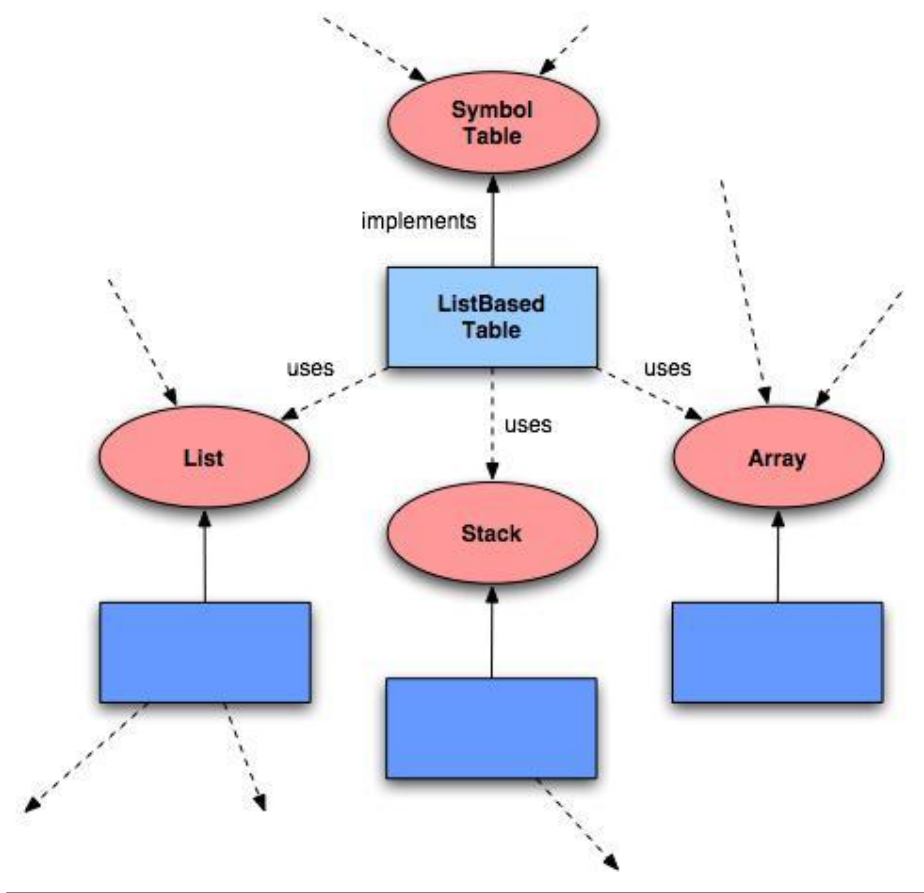
public void setSize(int x, int y);
requires nothing
ensures after the call, the ellipse is
x units wide and y units high



public void setSize(int x, int y);
requires $x = y$
ensures after the call, the ellipse is
x units wide and y units high

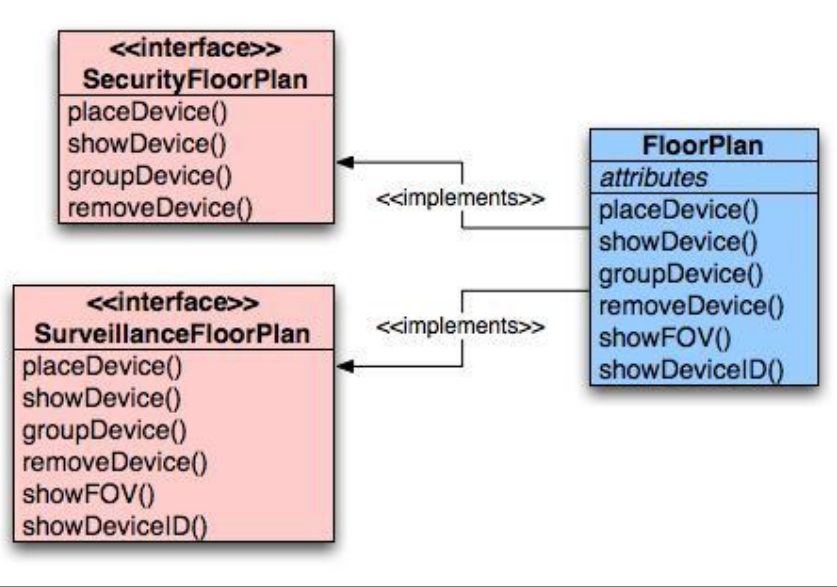
4. Dependency Inversion

- Depend on abstractions. Do not depend on concretions.



5.Interface Segregation

- Many client-specific interfaces are better than one general purpose interface.

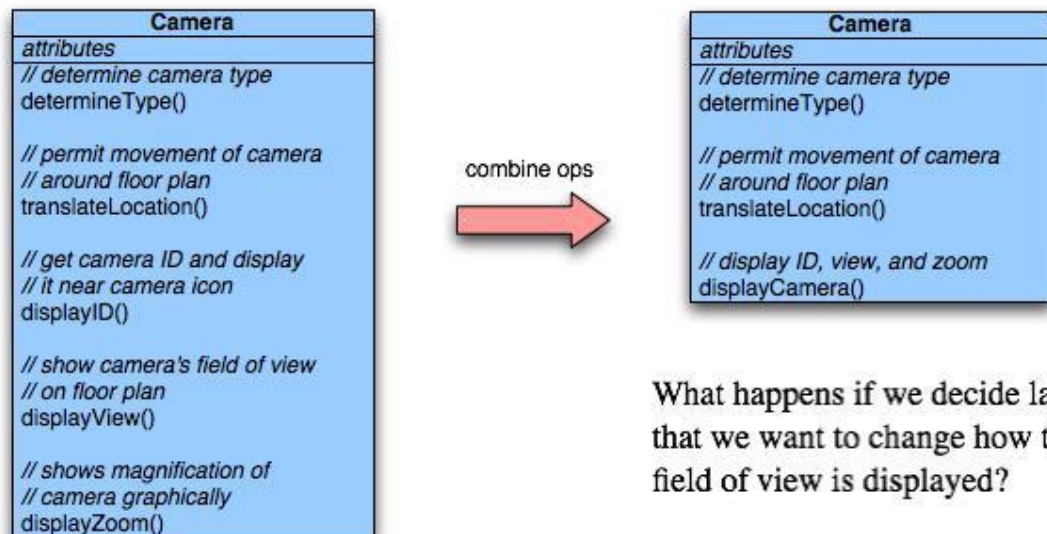


Cohesion

- The “**single-mindedness**” of a module.
- cohesion implies that a **single component or class encapsulates** only **attributes and operations** that are closely related to one another and to the class or component itself.
- Examples of cohesion
 - i) **Functional**
 - ii) **Layer**
 - iii) **Communicational**

i) Functional Cohesion

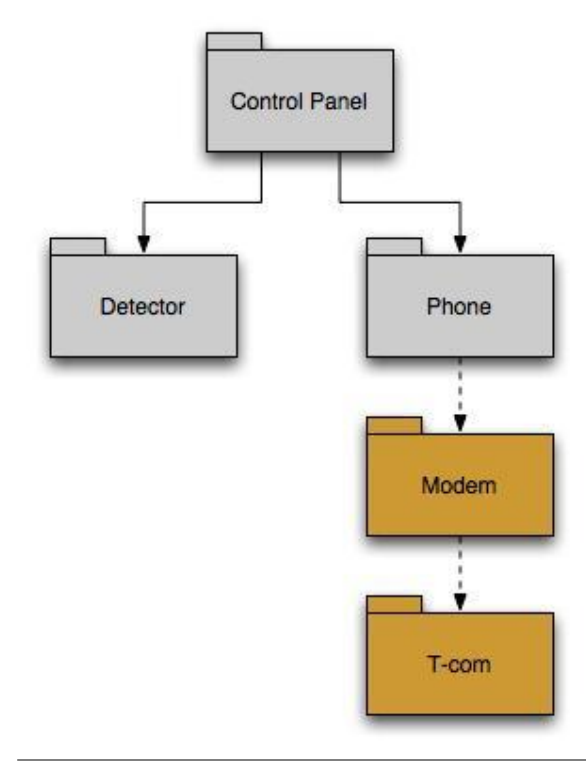
- Typically applies to operations. Occurs when a module performs one and only one computation and then returns a result.



What happens if we decide later that we want to change how the field of view is displayed?

ii) Layer Cohesion

- Applies to **packages, components, and classes.**
- Occurs when a higher layer can access a lower layer, but lower layers do not access higher layers.



iii) Communicational Cohesion

- All operations that access the same data are defined within one class.
- In general, such classes focus solely on the data in question, accessing and storing it.
- Example: A **StudentRecord** class that **adds, removes, updates, and accesses** various fields of a student record for client components.

Coupling

- A qualitative measure of the degree to which classes or components are connected to each other.
- Avoid
 - Content coupling
- Use caution
 - Common coupling
- Be aware
 - Routine call coupling
 - Type use coupling
 - Inclusion or import coupling

i) Content Coupling

- Occurs when one component “surreptitiously modifies data that is internal to another component”.
- Violates information hiding.
- What’s wrong here? →

```
public class StudentRecord {  
  
    private String name;  
    private int[ ] quizScores;  
  
    public String getName() {  
        return name;  
    }  
    public int getQuizScore(int n) {  
        return quizScores[n];  
    }  
    public int[ ] getAllQuizScores() {  
        return quizScores;  
    }  
  
    ....  
}
```

ii) Common Coupling

- Occurs when a number of components all make use of a global variable.

```
package FarWest;

import Environment.setup;

public class MyClass {

    public void doSomething() {
        // do something
        // use setup
    }

    public void doSomethingElse() {
        // do something else
        // modify setup
    }

    ...
}
```

```
package FarEast;

import Environment.setup;

public class YourClass {

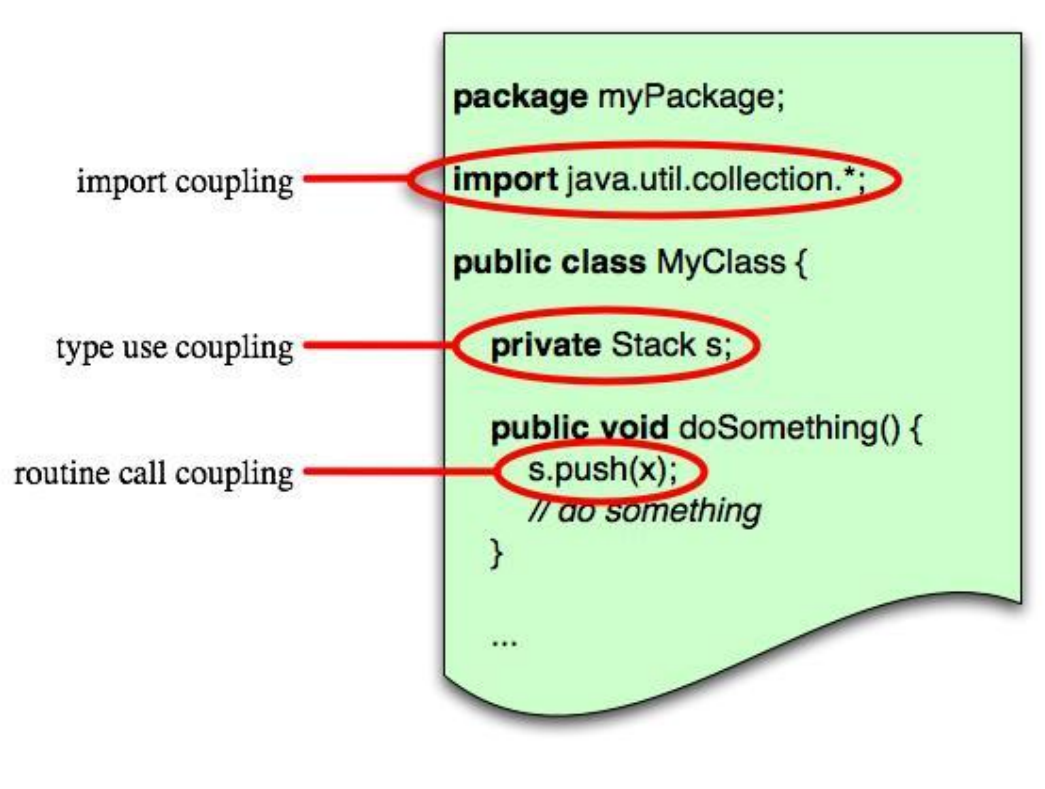
    public void doSomething() {
        // do something
        // use setup
    }

    public void doSomethingElse() {
        // do something else
        // modify setup
    }

    ...
}
```

iii) Routine Coupling

- Certain types of coupling occur routinely in object-oriented programming.



Component-Level Design

1. Identify **design classes** in problem domain.
2. Identify **infrastructure design classes**.
3. **Elaborate design classes.**
4. Describe **persistent data sources**.
5. **Elaborate behavioral representations.**
6. **Elaborate deployment diagrams.**
7. **Refactor design and consider alternatives.**

Steps 1 & 2 – Identify Classes

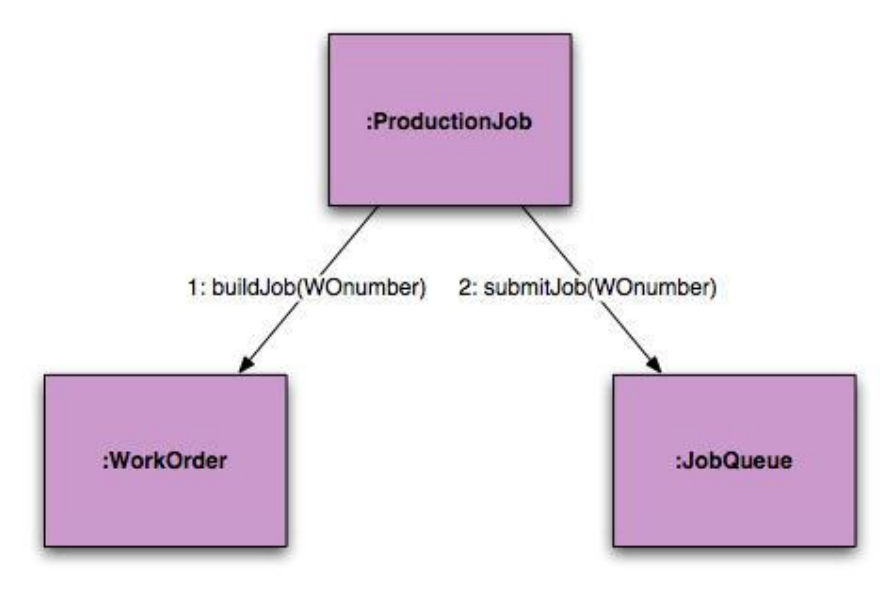
1. Most classes from the **problem domain** are **analysis classes** created as part of the **analysis model**.
2. The **infrastructure design classes** are introduced as **components** during **architectural design**.

Step 3 – Class Elaboration

- a) Specify **message details** when classes or components collaborate.
- b) Identify **appropriate interfaces** for each component.
- c) **Elaborate attributes** and **define data structures** required to implement them.
- d) Describe **processing flow** within each operation in detail.

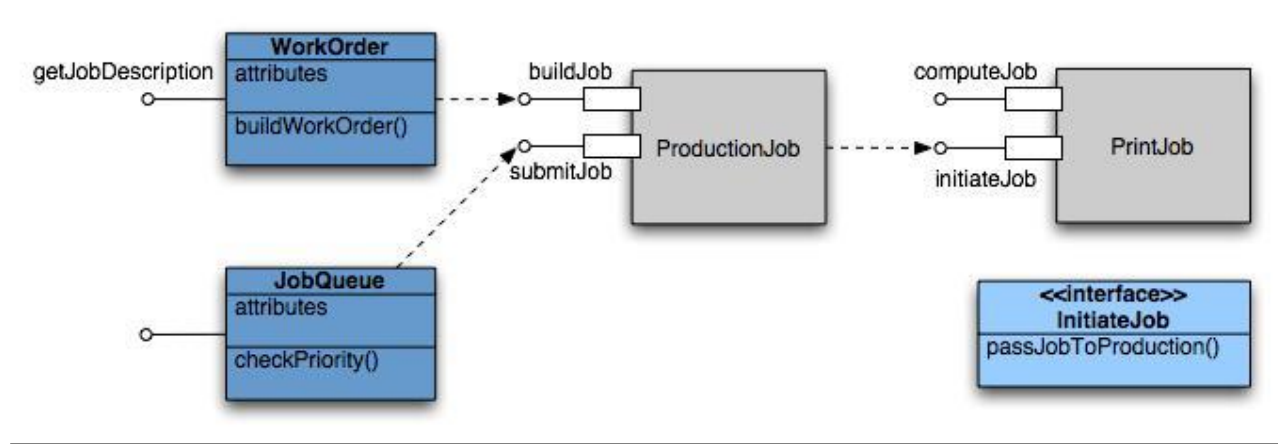
3a. Collaboration Details

- Messages can be elaborated by expanding their syntax in the following manner:
 - **[guard condition] sequence expression (return value) :=
message name (argument list)**



3b. Appropriate Interfaces

- Pressman argues that the PrintJob interface “initiateJob” in slide 5 does not exhibit sufficient cohesion because it performs three different subfunctions. He suggests this refactoring.

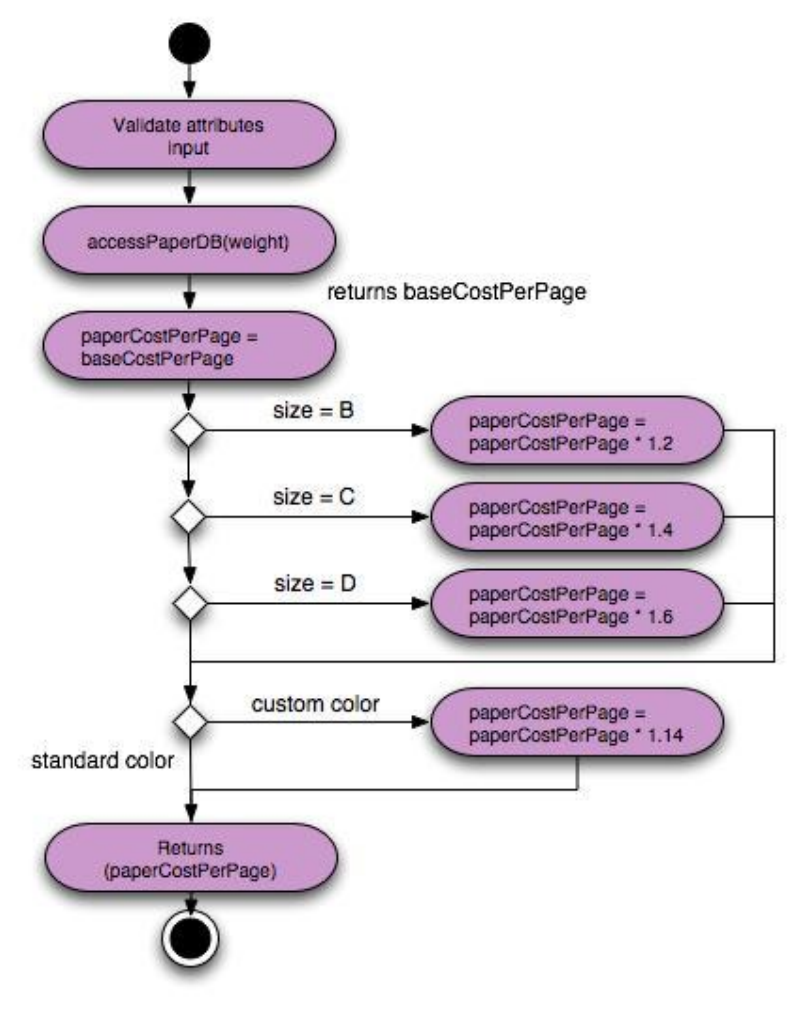


3c. Elaborate Attributes

- Analysis classes will typically only list names of general attributes (ex. paperType).
- List all attributes during component design.
- UML syntax:
 - **name : type-expression = initial-value { property string }**
- For example, paperType can be broken into weight, size, and color. The weight attribute would be:
 - **paperType-weight: string =“A” { contains 1 of 4 values – A, B, C, or D }**

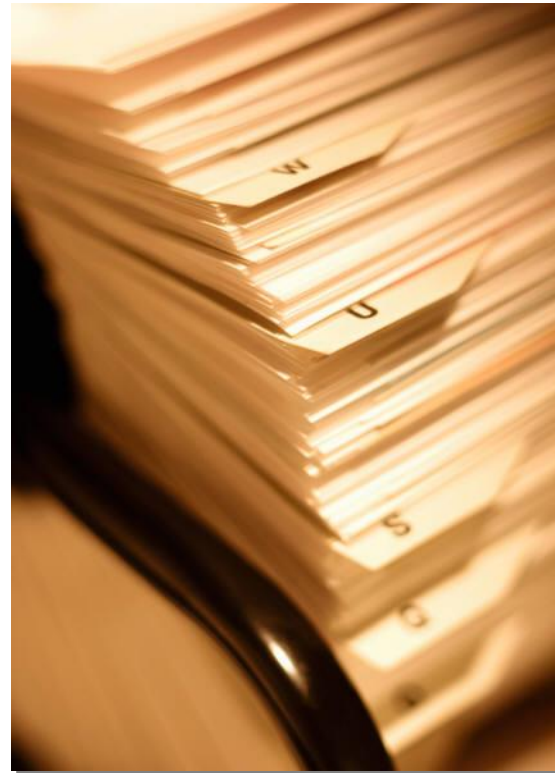
3d. Describe Processing Flow

- Activity diagram for `computePaperCost( )`



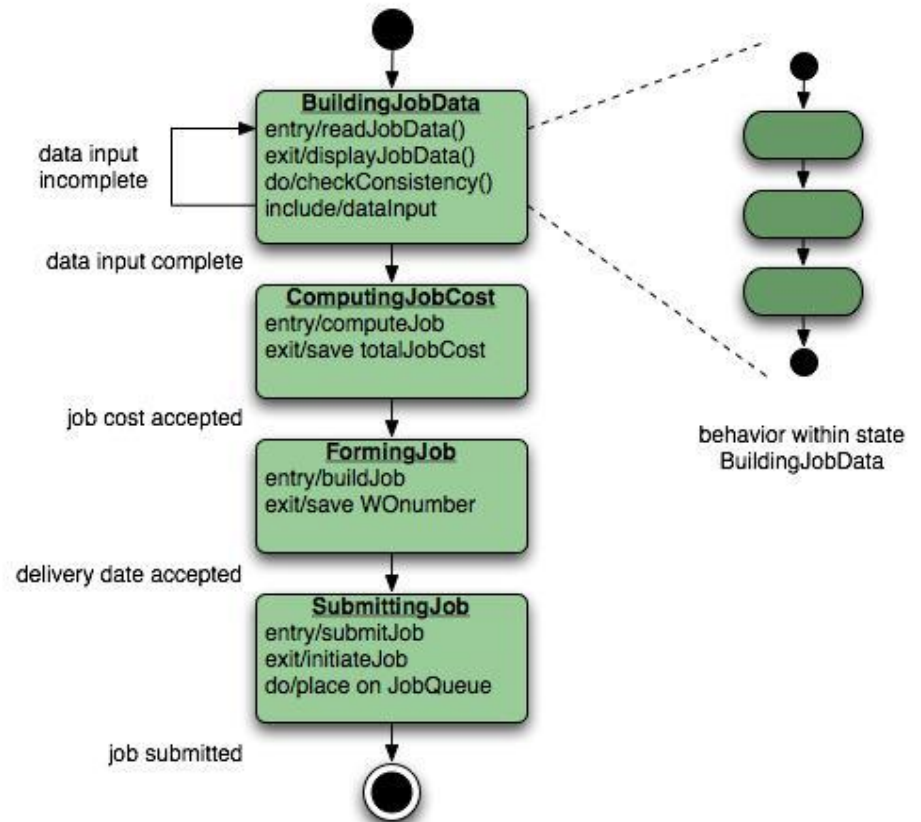
Step 4 – Persistent Data

- Describe persistent data sources (databases and files) and identify the classes required to manage them.



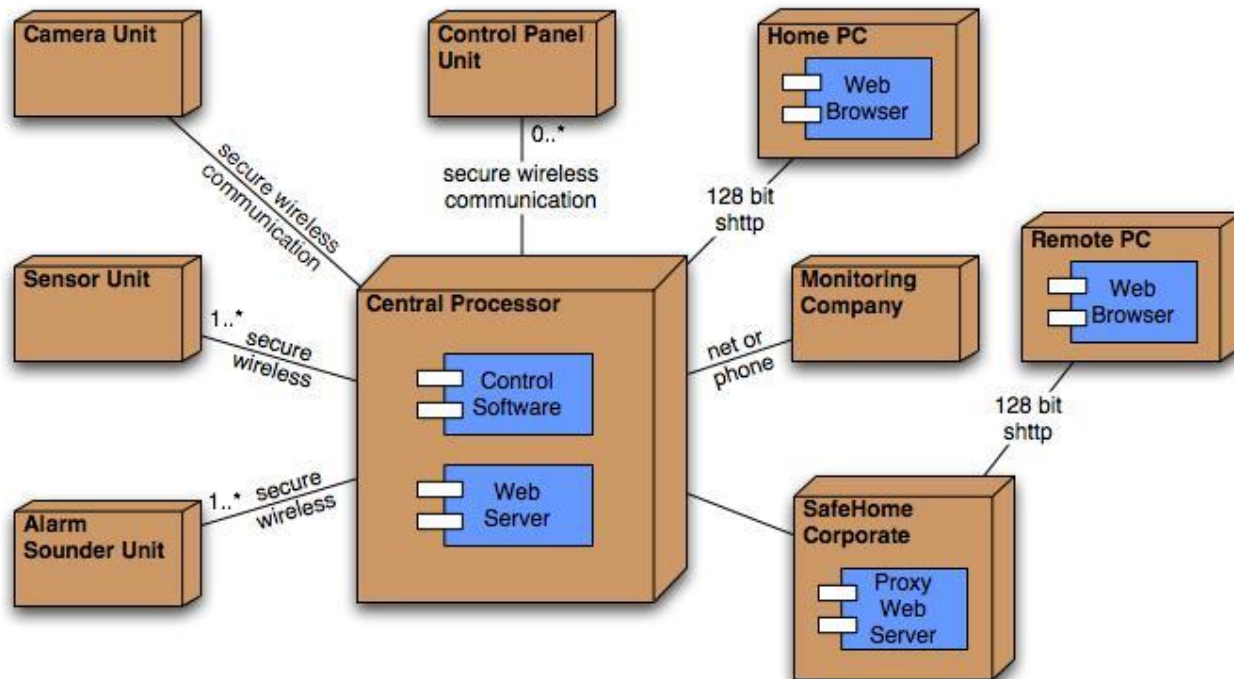
Step 5 – Elaborate Behavior

- It is sometimes necessary to model the behavior of a design class.
- Transitions from state to state have the form:
 - Event-name**
(parameter-list) [guard-condition] / action expression



Step 6 – Elab. Deployment

- Deployment diagrams are elaborated to represent the location of key packages or components.



Step 7 – Redesign/Reconsider

- The first component-level model you create will not be as **complete, consistent, or accurate** as the n^{th} **iteration** you apply to the model.
- The best designers will consider many **alternative design solutions** before settling on the final design model.

one in which all data elements and procedures of a module work together to achieve some objective. High cohesion is often characterized by more understanding, modifiability, and maintainability of the modules in a system. Low cohesion modules are highly undesirable and should be modified or replaced to meet the objectives of modular design. An important goal of designers is to maximize cohesion and minimize coupling. Cohesion between the elements of a module is measured in terms of the strength of the binding of the elements within the module itself. A functionally-independent module has higher cohesion as compared to dependent modules. An illustration of module cohesion is given in Figure 6.6.

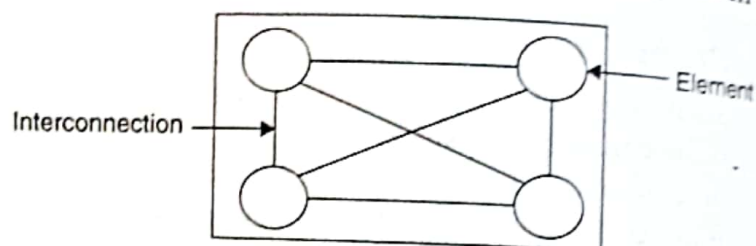


Figure 6.6 Module cohesion

There are many different levels of cohesion used at several levels of a module, such as functional cohesion, sequential cohesion, communicational cohesion, procedural cohesion, temporal cohesion, logical cohesion, and coincidental cohesion. Functional cohesion is the strongest (best) and coincidental cohesion is the weakest (worst) level of cohesion. The ratings of the common categories of cohesion ranked from the strongest (best) to the weakest (worst) are as follows:

1. Functional cohesion
2. Sequential cohesion
3. Communicational cohesion
4. Procedural cohesion
5. Temporal cohesion
6. Logical cohesion
7. Coincidental cohesion

Strongest (best)

Weakest (worst)

1. Functional Cohesion

Functional cohesion is the strongest cohesion as compared to other levels. In a functionally cohesive module, all the elements of the module perform a single function. It is a highly refined module that may rarely contain its sub-modules. These can be mathematical functions or functions having a well-defined goal. For example, "log" computes the logarithm of a number and "printf" prints the results.

2. Sequential Cohesion

Sequential cohesion exists when the output from one element of a module becomes the input for some other element. It may contain several functions that may perform different tasks for a single module. For example, "withdraw money" and "update balance" both are bound together to withdraw money from an account.

3. Communicational Cohesion

In communicational cohesion, all the elements of a module operate on the same input or output data. Here, a module may perform more than one function. For example, "print and punch the output file" can be communicational cohesion. The binding of elements in a module is higher than procedural cohesion.

4. Procedural Cohesion

Procedural cohesion contains the elements which belong to a common procedural unit. The functions are executed in a certain order. For example, "entering, reading, and verifying the ATM password" are bound in an ordered manner for the procedurally cohesive module "enter password."

5. Temporal Cohesion

Sometimes a module performs several functions in a sequence but their execution is related to a certain time. All the functions that are activated at a single time, such as start up or shut down, are brought together. For example, "a database trigger is activated on executing a certain procedure."

6. Logical Cohesion

Logical cohesion exists when logically-related elements of a module are placed together. The elements of a module that perform similar functions such as input, error handling, etc., are put together in a single module. A logically cohesive module integrates various related sub-modules and its subparts. All the parts communicate with each other by passing control information such as flag variable, using some shared source code, etc.

7. Coincidental Cohesion

Coincidental cohesion is the weakest level of cohesion. It occurs when the elements within a given module have no meaningful relationship to each other. Different parts of a module are not related but simply combined into a single module. If there is a change in one part, then there will be no effect on the other part of the module. Coincidental cohesion is possible through modularization of a module into several pieces.

SOFTWARE ARCHITECTURE

Software architecture is an evolving discipline in the modern society of software development. Most of the earlier designs have been focused on data structures and algorithms. Nowadays, the size and complexity of software systems have been expanded from standalone systems to distributed systems to handheld software systems. Therefore, various other design issues are needed to be considered for the development of software. These can be subsystem structures, control structures, communication between subsystems and their synchronization, data access, design of data structures, performance issues, physical distribution, selection of

There are various terms of modules, depending on the paradigm of the programming language. For example, a module can be a function, procedure, program, subroutine, class, package, framework, library files, templates, components, etc. A module has the following characteristics:^{9,10}

- Modularity measures the independency of the parts of a system and enhances separation of concerns.
- Modularity enhances quality factors, such as portability, extensibility, compatibility, scalability, etc.
- A module contains data structures, input/output statements, instructions, and processing logic.
- A module can be called into another module.
- A module can be reused within other modules.
- A modular system can be easily developed, maintained, and debugged.
- Modularity reduces design complexities in a system through distributed software architectures.
- Modularity uses abstraction, which helps in defining a subsystem.
- Modularity improves design clarity and understandability.

6.4 MODULAR DESIGN

A modular design focuses on minimizing the interconnections between modules. In a modular system design, several independent and executable modules are composed together to construct an executable application program. The programming language support, interfaces, and the information-hiding principles ensure modular system design. There are various modularization criteria to measure the modularity of a system. The most common criteria are functional independency; levels of abstraction; information hiding; functional diagrams, such as DFD, modular programming languages, coupling, and cohesion. *An effective modular system has low coupling and high cohesion.* Coupling and cohesion are often used as opposite ends of a scale in measuring how “good” a piece of software is. These metrics are commonly used for measuring the quality of an object-oriented code.

Coupling and cohesion are the most popular criteria used to measure the modularity in a system. In the following subsections, we will discuss coupling and cohesion in detail.

6.4.1 Coupling

Coupling is the strength of interconnection between modules. Alternatively, it is the measure of the degree of interdependency between modules. Modules are

either loosely coupled or strongly coupled. In strong coupling, two modules are dependent on each other. Strong coupling can be observed in assembly language programs where change in one part or data requires changes in other parts of the system. Also, these are difficult to reuse, test, and release for the operation. In loose coupling, there are weak interconnections between modules. Although modules are somehow interconnected with other modules, low coupling is generally preferred in good software design. Interdependency between modules increases as coupling increases. In Figure 6.5, modules M1 and M2 are coupled modules. Module M1 is dependent on module M2 and vice versa.

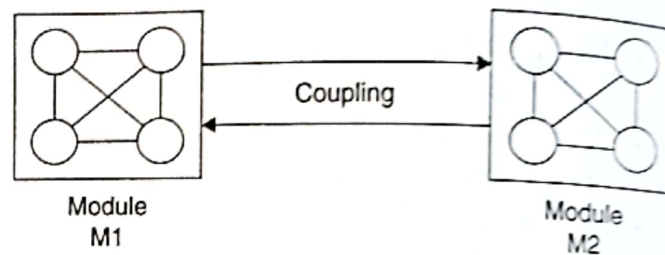


Figure 6.5 Coupled modules

Coupling increases as interconnection between modules increases. Interconnection between modules can be measured by the number of function calls, number of parameters passed, return values, data types, sharing of data files or data items, etc. The strength of interconnection between modules is influenced by the level of complexity of the interfaces, type of connection, and the type of communication. The complexity of an interface is the measure of the type of parameters, number of parameters, common sharing of data or code communicated by modules. The connection between modules is established by relating one module to another through parts of the system or through certain data value. Communication is the data passed, type of data, and the control information passed to another module.

There are different types of coupling between modules, such as message coupling, data coupling, stamp coupling, control coupling, external coupling, common coupling, and content coupling. Message coupling is the lowest (best) and content coupling is the highest (worst) level of coupling. The ranking of the coupling from lowest (best) to highest (worst) is as follows:

1. Message coupling
2. Data coupling
3. Stamp coupling
4. Control coupling
5. External coupling
6. Common coupling
7. Content coupling

Lowest (best)

Highest (worst)

1. Message Coupling

Message coupling is the lowest (i.e., best) type of coupling which exists between modules. It can be observed in an object-oriented or a component-based system.

where objects or components communicate through parameters or message passing. For example, in C++ objects communicate with each other by sending a message through parameters in the function call.

2. Data Coupling

Data coupling exists between modules when data are passed as parameters in the argument list of the function call. Each datum is a primary data item (e.g., integer, character, float, etc.) that can be used between modules. Data coupling is good if a small number of data items is passed in the function call.

3. Stamp Coupling

Stamp coupling occurs between modules when data are passed by parameters using complex data structures, which may use parts or the entire data structure by other modules. For example, structures in C, records in Pascal, etc. Stamp coupling is preferred when most of the items are used in the argument list of the function.

4. Control Coupling

Control coupling exists when one module controls the flow of another by passing control information such as flag set or switch statements. Control coupling controls the sequence of instruction execution in another module. For example, a flag variable decides what function or module is to be executed next.

5. External Coupling

External coupling occurs when two modules share an externally imposed data format, communication protocol, or device interface. All the modules share the same I/O device or the external environment.

6. Common Coupling

Common coupling is when two modules share common data (e.g., a global variable). In C language, external data items are accessed by all modules in the program. If there is any change in the shared resource, it influences all the modules using it.

7. Content Coupling

It is the highest coupling (worst). Content coupling exists between two modules when one module refers or shares its internal working with another module. Accessing local data items or instructions of another module is an example of content coupling.

42 Cohesion

Cohesion of a single module is the degree to which the elements of a single module are functionally related to achieve an objective. Module cohesion represents how tightly bound the internal elements of the module are to one another. Higher cohesion is preferred for an effective modular design. Generally, whenever cohesion is greater, coupling between the modules is lower. A highly cohesive system is

Chapter 10

Architectural Design

Software Engineering: A Practitioner's Approach, 6th edition
by Roger S. Pressman

Software Architecture

▪ The software architecture of a program or computing system is:

- the **structure or structures** of the system,
- comprises the **software components**,
- **externally visible properties** of those **components**, and
- the **relationships** among them.

— *Bass. et al.*

Why Architecture?

- Architecture is a representation of a system that enables the software engineer to:
 1. analyze the effectiveness of the design in meeting its stated requirements,
 2. consider architectural alternatives at a stage when making design changes is still relatively easy, and
 3. reduce the risks associated with the construction of the software.

Data Design

- **Architectural level** → Database design
 - data mining
 - data warehousing
- **Component level** → Data structure design

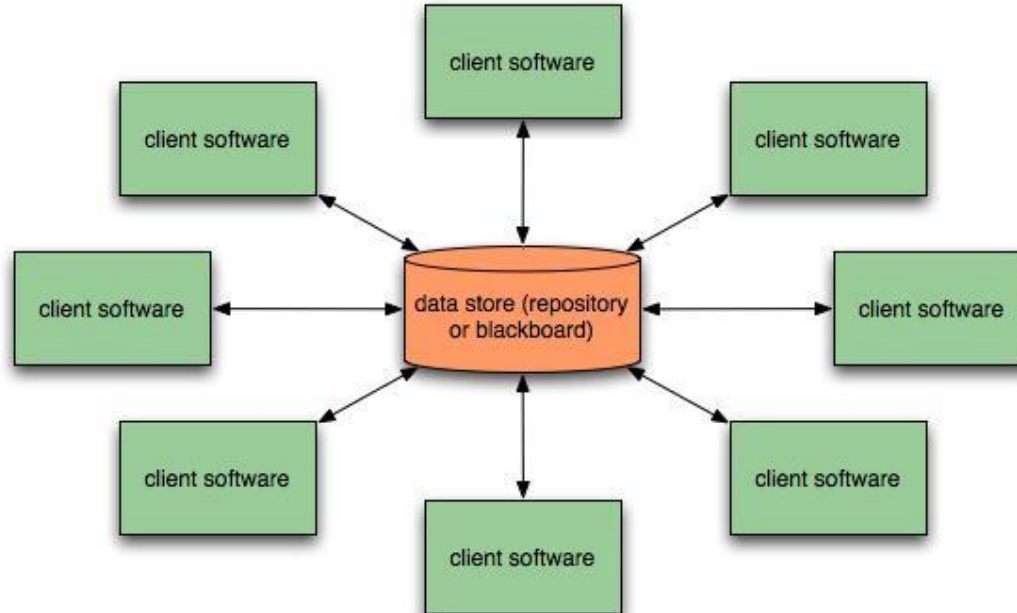
Architectural Styles

- Each style describes a system category that encompasses:
 1. **a set of components** (e.g., a database, computational modules) that perform a function required by a system,
 2. **a set of connectors** that enable “communication, coordination, and cooperation” among components,
 3. **constraints** that define how components can be integrated to form the system, and
 4. **semantic models** that enable a designer to understand the overall properties of a system.

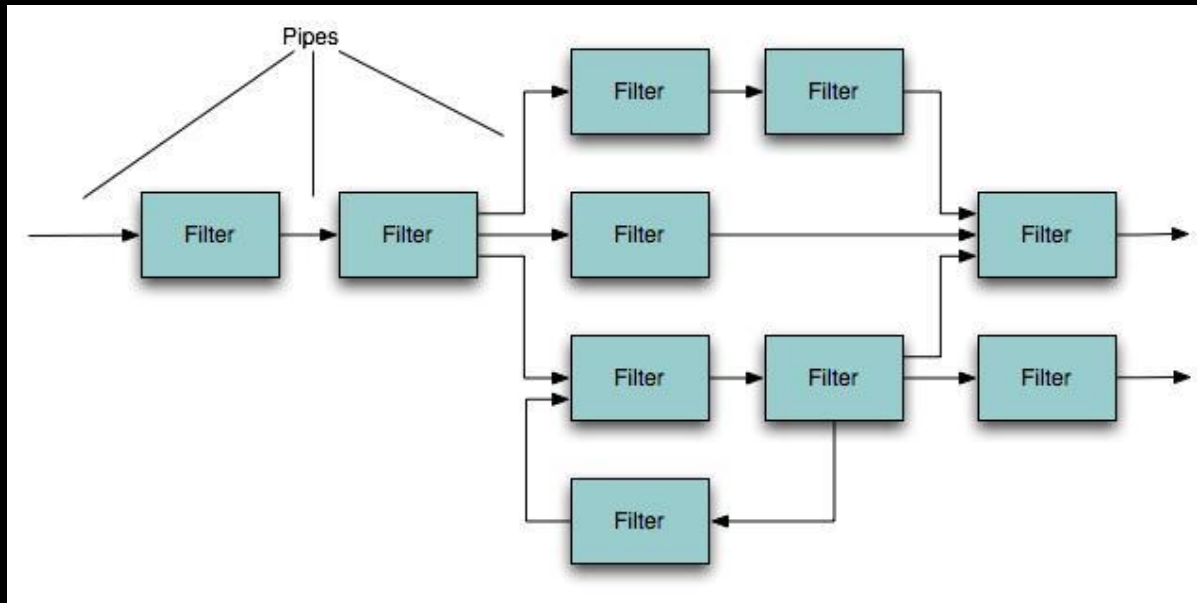
Specific Styles

1. **Data-centered architecture.**
2. **Data flow architecture.**
3. **Call and return architecture.**
4. **Object-oriented architecture.**
5. **Layered architecture.**

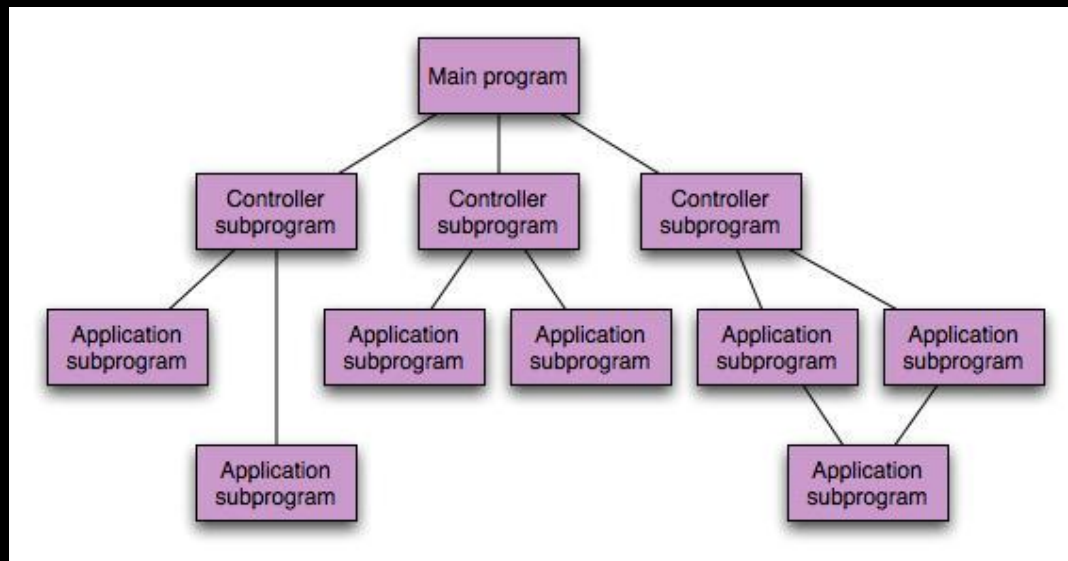
1.Data-Centered Architecture



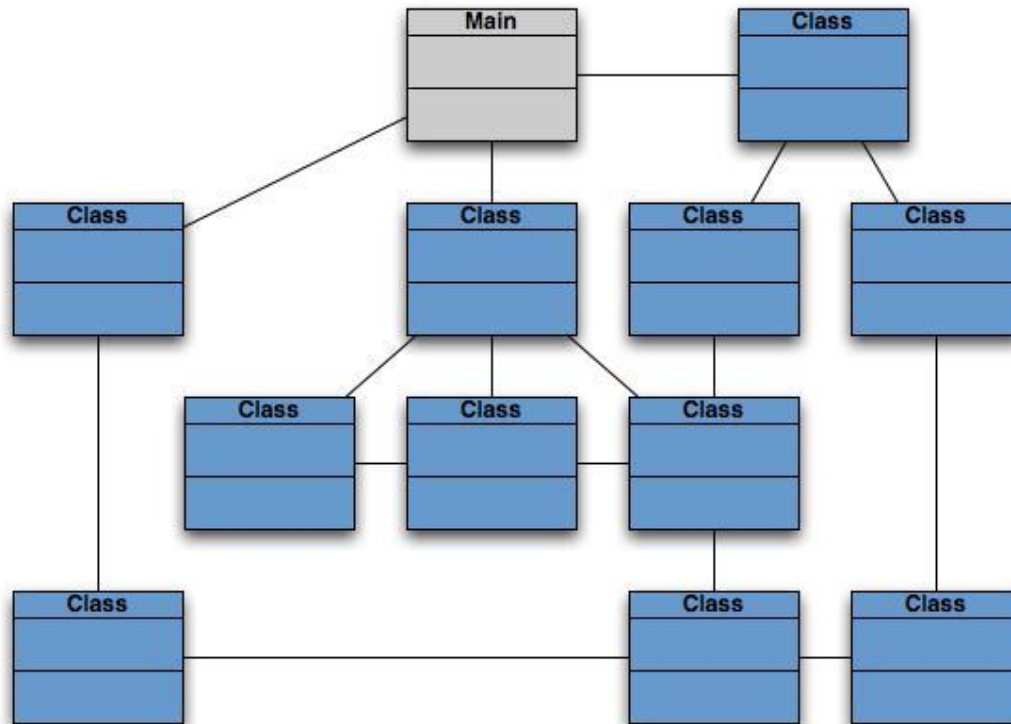
2.Data-Flow Architecture



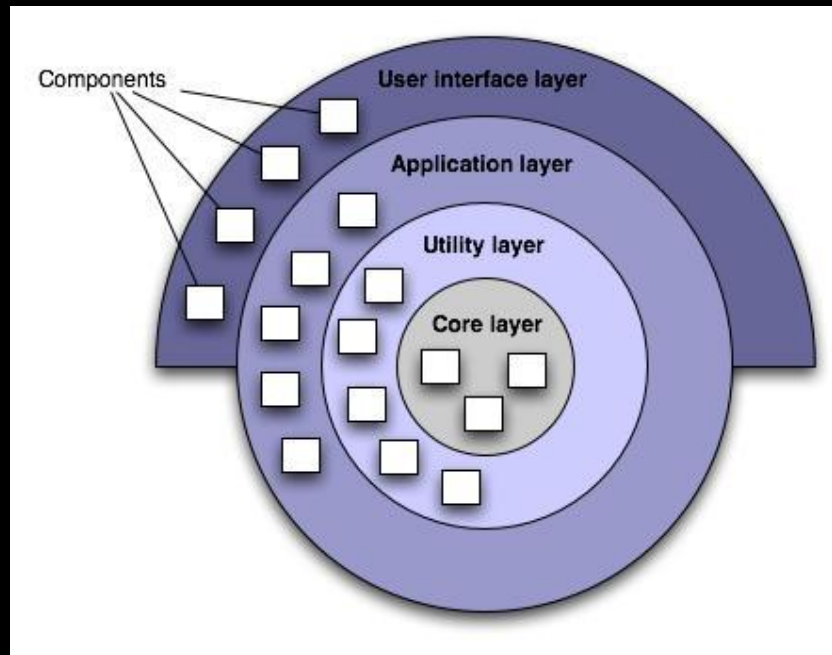
3.Call and Return Architecture



4.Object-Oriented Architecture



5.Layered Architecture



Architectural Patterns

▪ Concurrency

- operating system process management
- task scheduler

▪ Persistence

- database management system
- application level persistence

▪ Distribution

- broker

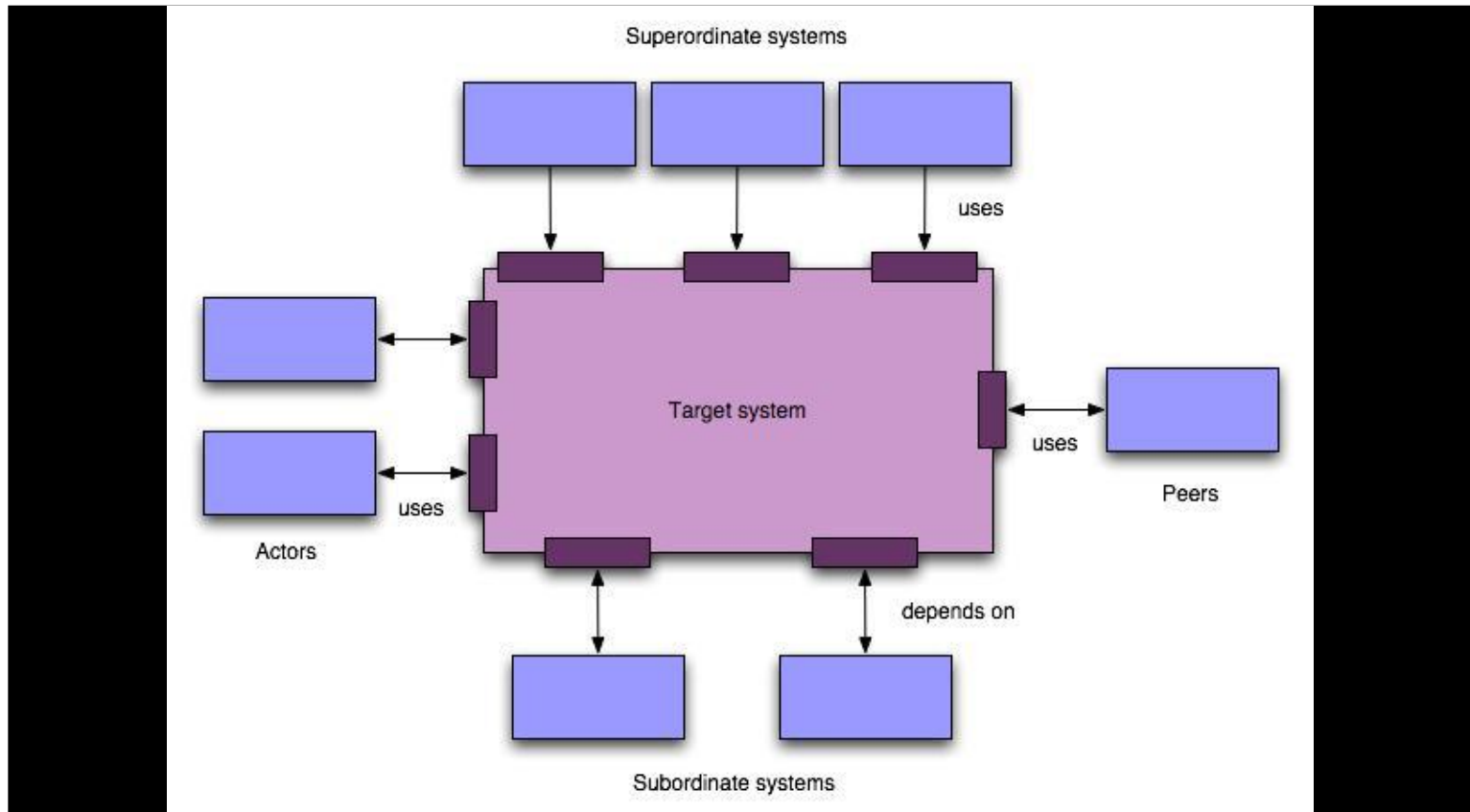
Architectural Design

1. Architectural context diagrams-model how software interacts with external entities.

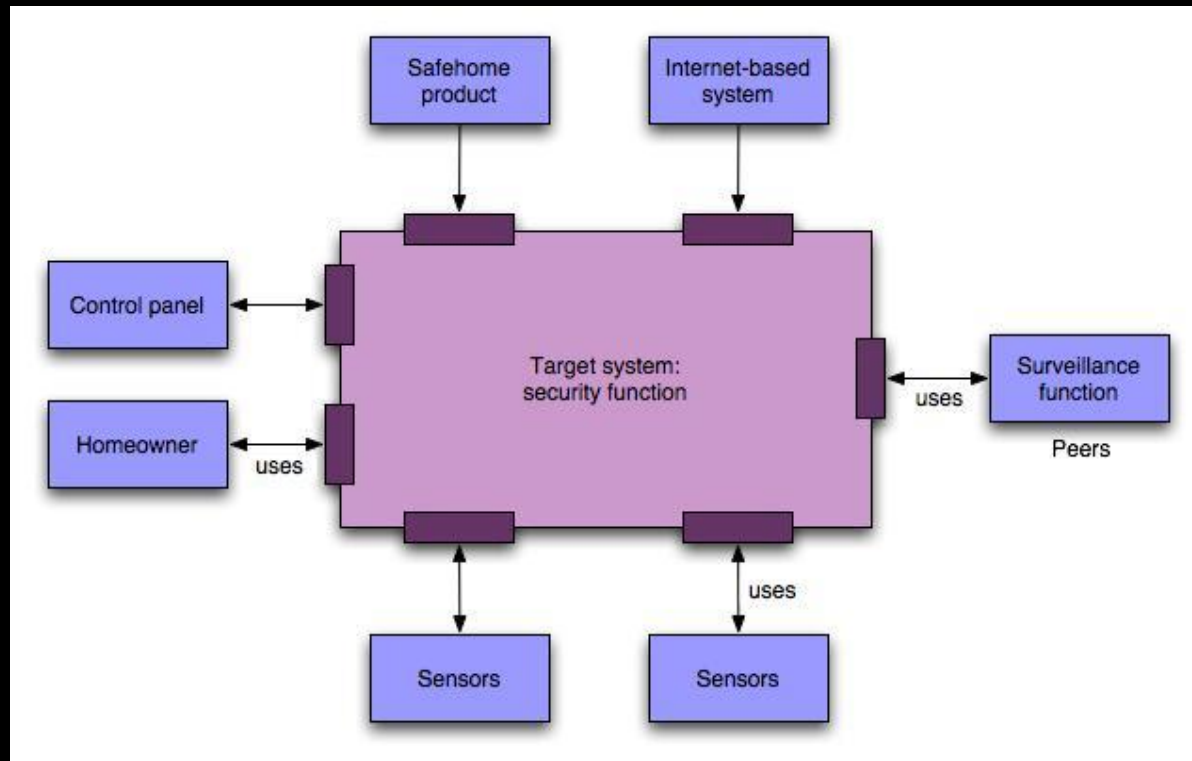
2. Archetypes-classes or patterns that represent an abstraction critical to the system.

3. Architectural components-derived from the application domain, the infrastructure, and the interface.

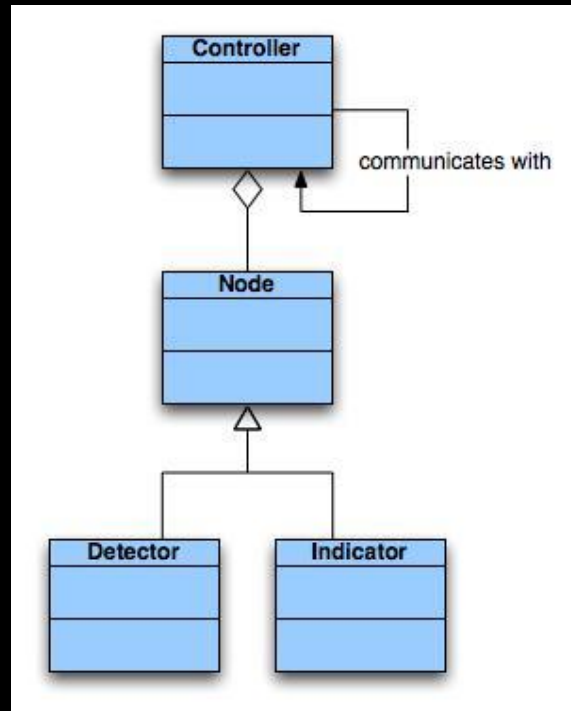
Arch. Context Diagram



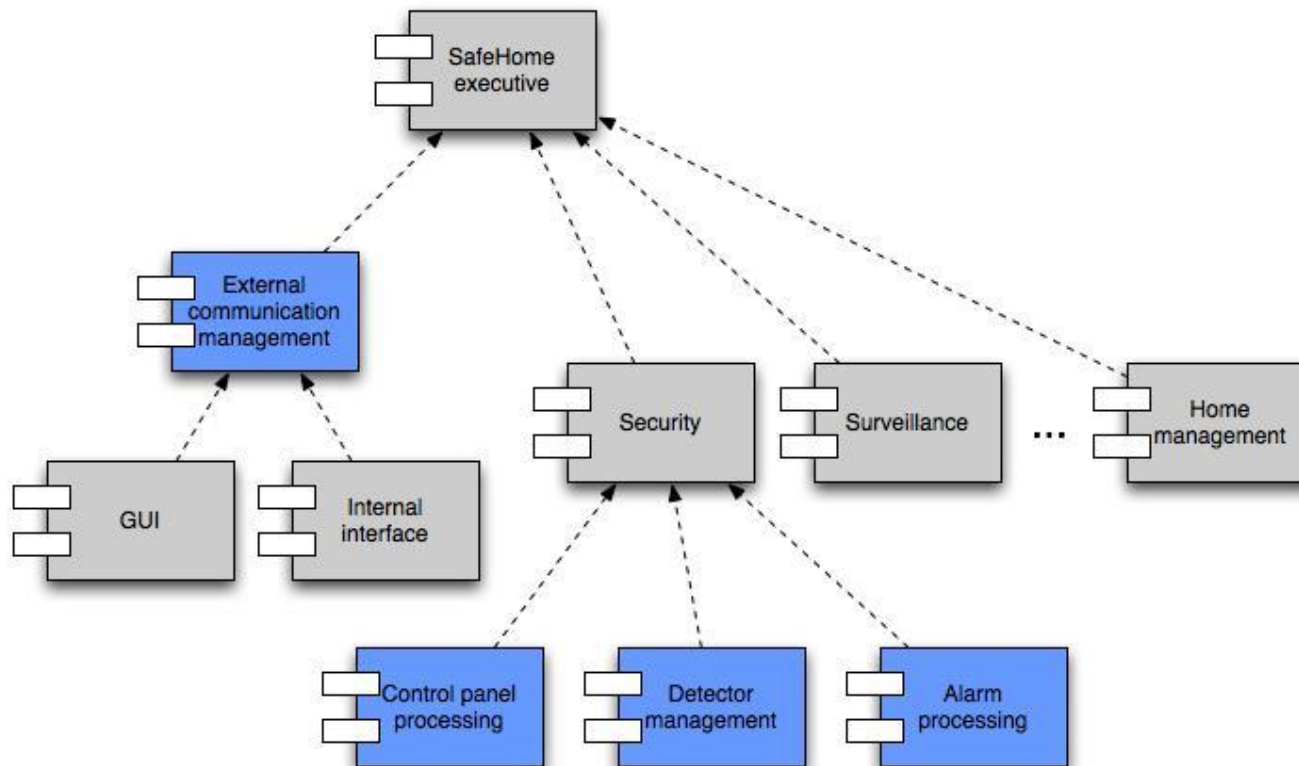
SafeHome ACD



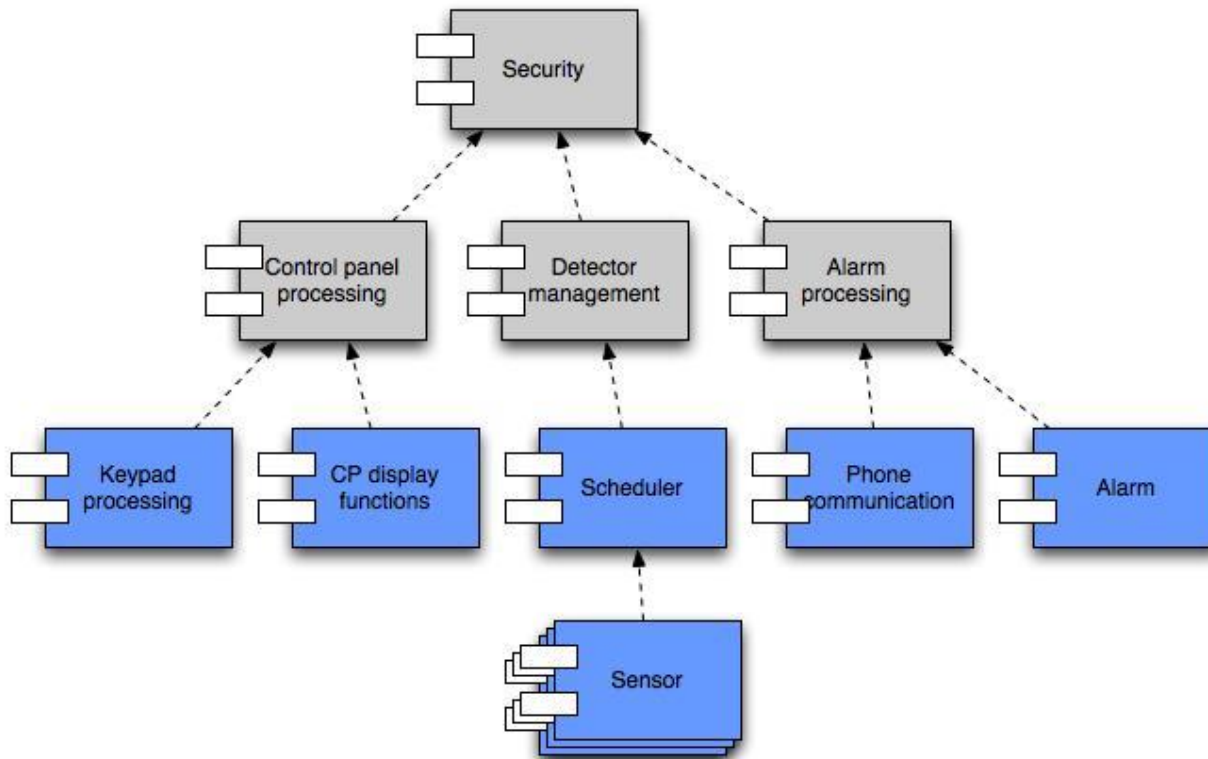
SafeHome Archetype



Component Structure



Component Elaboration



Chapter 4

Principles that Guide Practice

Software Engineering: A Practitioner's Approach, 7/e

by Roger S. Pressman

Software Engineering Knowledge

- You often hear people say that software development knowledge has a 3-year half-life: half of what you need to know today will be obsolete within 3 years. In the domain of technology-related knowledge, that's probably about right.
- But there is another kind of software development knowledge —a kind that I think of as "software engineering principles" — that does not have a three-year half-life. These software engineering principles are likely to serve a professional programmer throughout his or her career.

Steve McConnell

What, who, why?

- Software Practice is a broad array of principles, concepts, methods and tools that you must consider as software is planned and developed.
- Software engineers and their managers conduct a variety of software engineering tasks are using it.
- Software Process provides everyone with a road map for getting to a successful destination. Practice provides you with the details you will need to drive along the road. Where the bridges, the roadblocks, and the forks are located? It instructs you how to drive, where to slow down, and where to speed up. In the software engineering context, it is what you do day in and day out as software evolves from an idea to a reality.
- Three elements: principles, concepts and methods. A fourth element namely tools supports the application of methods.

General Principles that Span Software Engineering

Process and Practice High Level Abstraction

- Principle #1. *Provide Value to end users.*
- Principle #2. *Keep it simple*
- Principle #3. *Maintain the vision for the product and project.*
- Principle #4. *Recognize that other consume what you produce*
- Principle #5. *Be open to the future*
- Principle #6. *Plan ahead for reuse*
- Principle #7. *Think!*

1.Principles that Guide Process(8)

- Principle #1. *Be agile*. Whether the process model you choose is prescriptive or agile, the basic tenets of agile development should govern your approach.
- Principle #2. *Focus on quality at every step*. The exit condition for every process activity, action, and task should focus on the quality of the work product that has been produced.
- Principle #3. *Be ready to adapt*. Process is not a religious experience and dogma has no place in it. When necessary, adapt your approach to constraints imposed by the problem, the people, and the project itself.
- Principle #4. *Build an effective team*. Software engineering process and practice are important, but the bottom line is people. Build a self-organizing team that has mutual trust and respect.

1.Principles that Guide Process(cont..)

- Principle #5. Establish mechanisms for communication and coordination.

Projects fail because important information falls into the cracks and/or stakeholders fail to coordinate their efforts to create a successful end product.

- Principle #6. Manage change. The approach may be either formal or informal, but mechanisms must be established to manage the way changes are requested, assessed, approved and implemented.

- Principle #7. Assess risk. Lots of things can go wrong as software is being developed. It's essential that you establish contingency plans.

- Principle #8. Create work products that provide value for others. Create only those work products that provide value for other process activities, actions or tasks.

2.Principles that Guide Practice(8)

- Principle #1. *Divide and conquer*. Stated in a more technical manner, analysis and design should always emphasize *separation of concerns* (SoC).
- Principle #2. *Understand the use of abstraction*. At its core, an abstraction is a simplification of some complex element of a system used to communicate meaning in a single phrase.
- Principle #3. *Strive for consistency*. A familiar context makes software easier to use.
- Principle #4. *Focus on the transfer of information*. Pay special attention to the analysis, design, construction, and testing of interfaces.

2.Principles that Guide Practice(cont..)

- Principle #5. *Build software that exhibits effective modularity.*

Separation of concerns (Principle #1) establishes a philosophy for software. *Modularity* provides a mechanism for realizing the philosophy.

- Principle #6. *Look for patterns.* Brad Appleton [App00] suggests that: “The goal of patterns within the software community is to create a body of literature to help software developers resolve recurring problems encountered throughout all of software development.

- Principle #7. *When possible, represent the problem and its solution from a number of different perspectives.*

- Principle #8. *Remember that someone will maintain the software.*

3.Communication Principles(10)

- Principle #1. Listen. Try to focus on the speaker's words, rather than formulating your response to those words.
- Principle # 2. Prepare before you communicate. Spend the time to understand the problem before you meet with others.
- Principle # 3. Someone should facilitate the activity. Every communication meeting should have a leader (a facilitator) to keep the conversation moving in a productive direction; (2) to mediate any conflict that does occur, and (3) to ensure that other principles are followed.
- Principle #4. Face-to-face communication is best. But it usually works better when some other representation of the relevant information is present.

3.Communication Principles(cont..)

- Principle # 5. Take notes and document decisions. Someone participating in the communication should serve as a “recorder” and write down all important points and decisions.
- Principle # 6. Strive for collaboration. Collaboration and consensus occur when the collective knowledge of members of the team is combined ...
- Principle # 7. Stay focused, modularize your discussion. The more people involved in any communication, the more likely that discussion will bounce from one topic to the next.
- Principle # 8. If something is unclear, draw a picture.
- Principle # 9. (a) Once you agree to something, move on; (b) If you can't agree to something, move on; (c) If a feature or function is unclear and cannot be clarified at the moment, move on.
- Principle # 10. Negotiation is not a contest or a game. It works best when both parties win.

4.Planning Principles(10)

- Principle #1. Understand the scope of the project. It's impossible to use a roadmap if you don't know where you're going. Scope provides the software team with a destination.
- Principle #2. Involve the customer in the planning activity. The customer defines priorities and establishes project constraints.
- Principle #3. Recognize that planning is iterative. A project plan is never engraved in stone. As work begins, it very likely that things will change.
- Principle #4. Estimate based on what you know. The intent of estimation is to provide an indication of effort, cost, and task duration, based on the team's current understanding of the work to be done.

4.Planning Principles(cont..)

- Principle #5. Consider risk as you define the plan. If you have identified risks that have high impact and high probability, contingency planning is necessary.
- Principle #6. Be realistic. People don't work 100 percent of every day.
- Principle #7. Adjust granularity as you define the plan. Granularity refers to the level of detail that is introduced as a project plan is developed.
- Principle #8. Define how you intend to ensure quality. The plan should identify how the software team intends to ensure quality.
- Principle #9. Describe how you intend to accommodate change. Even the best planning can be obviated by uncontrolled change.
- Principle #10. Track the plan frequently and make adjustments as required. Software projects fall behind schedule one day at a time.

5. Modeling Principles

- In software engineering work, two classes of models can be created:
 - Requirements models (also called analysis models) represent the customer requirements by depicting the software in three different domains: the information domain, the functional domain, and the behavioral domain.
 - Design models represent characteristics of the software that help practitioners to construct it effectively: the architecture, the user interface, and component-level detail.

5.1:Requirements Modeling Principles(5)

- Principle #1. The information domain of a problem *must be represented and understood.*
- Principle #2. The functions that the software performs *must be defined.*
- Principle #3. The behavior of the software (as a *consequence of external events*) *must be represented.*
- Principle #4. The models that depict information, *function, and behavior must be partitioned in a manner that uncovers detail in a layered (or hierarchical) fashion.*
- Principle #5. The analysis task should move from *essential information toward implementation detail.*

5.2:Design Modeling Principles(9)

- Principle #1. *Design should be traceable to the requirements model.*
- Principle #2. *Always consider the architecture of the system to be built.*
- Principle #3. *Design of data is as important as design of processing functions.*
- Principle # 4. Interfaces (both internal and external) must be designed with care.
- Principle #5. User interface design should be tuned to the needs of the end-user. However, in every case, it should stress ease of use.
- Principle #6. Component-level design should be functionally independent.
- Principle #7. Components should be loosely coupled to one another and to the external environment.
- Principle #8. Design representations (models) should be easily understandable.
- Principle #9. The design should be developed iteratively. With each iteration, the designer should strive for greater simplicity.

6. Agile Modeling Principles(10)

- Principle #1. *The primary goal of the software team is to build software, not create models.*
- Principle #2. *Travel light—don't create more models than you need.*
- Principle #3. *Strive to produce the simplest model that will describe the problem or the software.*
- Principle #4. *Build models in a way that makes them amenable to change.*
- Principle #5. *Be able to state an explicit purpose for each model that is created.*
- Principle #6. *Adapt the models you develop to the system at hand.*
- Principle #7. *Try to build useful models, but forget about building perfect models.*
- Principle #8. *Don't become dogmatic about the syntax of the model. If it communicates content successfully, representation is secondary.*
- Principle #9. *If your instincts tell you a model isn't right even though it seems okay on paper, you probably have reason to be concerned.*
- Principle #10. *Get feedback as soon as you can.*

7.Construction Principles

- The construction activity encompasses a set of coding and testing tasks that lead to operational software that is ready for delivery to the customer or end-user.
- Coding principles and concepts are closely aligned programming style, programming languages, and programming methods.
- Testing principles and concepts lead to the design of tests that systematically uncover different classes of errors and to do so with a minimum amount of time and effort.

8.PreparationPrinciples(5)

■ Before you write one line of code, be sure you:

- Understand of the problem you're trying to solve.
- Understand basic design principles and concepts.
- Pick a programming language that meets the needs of the software to be built and the environment in which it will operate.
- Select a programming environment that provides tools that will make your work easier.
- Create a set of unit tests that will be applied once the component you code is completed.

9.Coding Principles(9)

■ As you begin writing code, be sure you:

- Constrain your algorithms by following structured programming practice.
- Consider the use of pair programming
- Select data structures that will meet the needs of the design.
- Understand the software architecture and create interfaces that are consistent with it.
- Keep conditional logic as simple as possible.
- Create nested loops in a way that makes them easily testable.
- Select meaningful variable names and follow other local coding standards.
- Write code that is self-documenting.
- Create a visual layout (e.g., indentation and blank lines) that aids understanding.

10.Validation Principles(3)

- After you've completed your first coding pass, be sure you:
 - Conduct a code walkthrough when appropriate.
 - Perform unit tests and correct errors you've uncovered.
 - Refactor the code.

11. Testing Principles(5)

- Al Davis [Dav95] suggests the following:
 - **Principle #1.** *All tests should be traceable to customer requirements.*
 - **Principle #2.** *Tests should be planned long before testing begins.*
 - **Principle #3.** *The Pareto principle applies to software testing.*
 - **Principle #4.** *Testing should begin “in the small” and progress toward testing “in the large.”*
 - **Principle #5.** *Exhaustive testing is not possible.*

12.Deployment Principles(5)

- **Principle #1.** *Customer expectations for the software must be managed.* Too often, the customer expects more than the team has promised to deliver, and disappointment occurs immediately.
- **Principle #2.** *A complete delivery package should be assembled and tested.*
- **Principle #3.** *A support regime must be established before the software is delivered.* An end-user expects responsiveness and accurate information when a question or problem arises.
- **Principle #4.** *Appropriate instructional materials must be provided to end-users.*
- **Principle #5.** *Buggy software should be fixed first, delivered later.*

CODING STANDARDS

Coding Standard

- **What are Coding Standards?**

Coding standards provide guidelines for programmers regarding naming, file organization, statements and declarations and layout and comments.

- **Why are Coding Standards needed?**

- Typical real-world projects are developed and maintained by teams of developers.
- Particularly in the context of open source projects, code is shared, updated and merged by many individuals across organizations and geographies.
- Naming conventions are therefore important.

Coding Standard (Cont.)

- There is no specific coding style or standard defined or there is no specific rule book.

Every organization has their own approaches!

Coding Standard (Cont.)

Then How do we decide Coding Standards?

TECH

www.edtechnology.in

CODING STANDARDS IN SOFTWARE ENGINEERING | SOFTWARE ENGINEERING LECTURES

Coding Standard (Cont.)

- Good software development organizations want their programmers to maintain to some **well-defined and standard style of coding** called **coding standards**
- So in order to achieve the same **they define and formulate rules and some of the basic things** that include are :

TECH

www.edtechnology.in

2:37 / 7:56

Coding Standard (Cont.)

- Naming Convention
- Files
- Commenting and Layout
- Limited use of global variables and Functions
- Indentation
- Exception Handling
- Statements



Coding Standard (Cont.)

- Naming Convention
 - Package names should be in lower case.
 - Type names should be nouns and should start with uppercase.
 - Variable names should be nouns starting with lowercase.
 - Constant names should be all uppercase.
 - Method names should verbs starting with lowercase.
 - Private class variables should have the `_`suffix.



Coding Standard (Cont.)

- **Files**

- Java source files should have the extension .java.
- Each file should contain one outer class and class name should be same as the file name.
- Line length should be limited to less than 80 columns and special characters should be avoided.

www.edtechnology.in



Coding Standard (Cont.)

- **Commenting and Layout**

- Single line comments for a block of code should be aligned with the code they are meant for.
- There should be comments for all major variables explaining what they represent.
- A block of comments should be preceded by a blank comment line with just “/*” and ended with a line containing just “*/”.
- Trailing comments after statements should be short, on the same line, and shifted far enough to separate them from statement.

www.edtechnology.in



Coding Standard (Cont.)

- **Limited Use of Global Variables and Functions**
 - These rules tell about which types of data that can be declared global and the data that can't be.
- **Appropriate Indentation**
 - Proper indentation makes code readability better.
 - There must be a space after giving a comma between two function arguments.
 - Each nested block should be properly indented and spaced.



Coding Standard (Cont.)

- **Exception Handling**
 - All errors or exceptions should return appropriate errors and before coding begins in some cases we need to plan possible exception and code their exception handling mechanisms.
- **Statements**
 - Class variables should never be declared public.
 - Avoid using Go to statement.
 - Loop variables should be initialized immediately before the loop.
 - Avoid the use of **break** and **continue** in a loop.
 - Avoid the use of **do.... while** constructs.

